

CS3.301: Operating Systems and Networks

Lecture 16 & 17 – Smaller Page Tables and Beyond Physical Memory

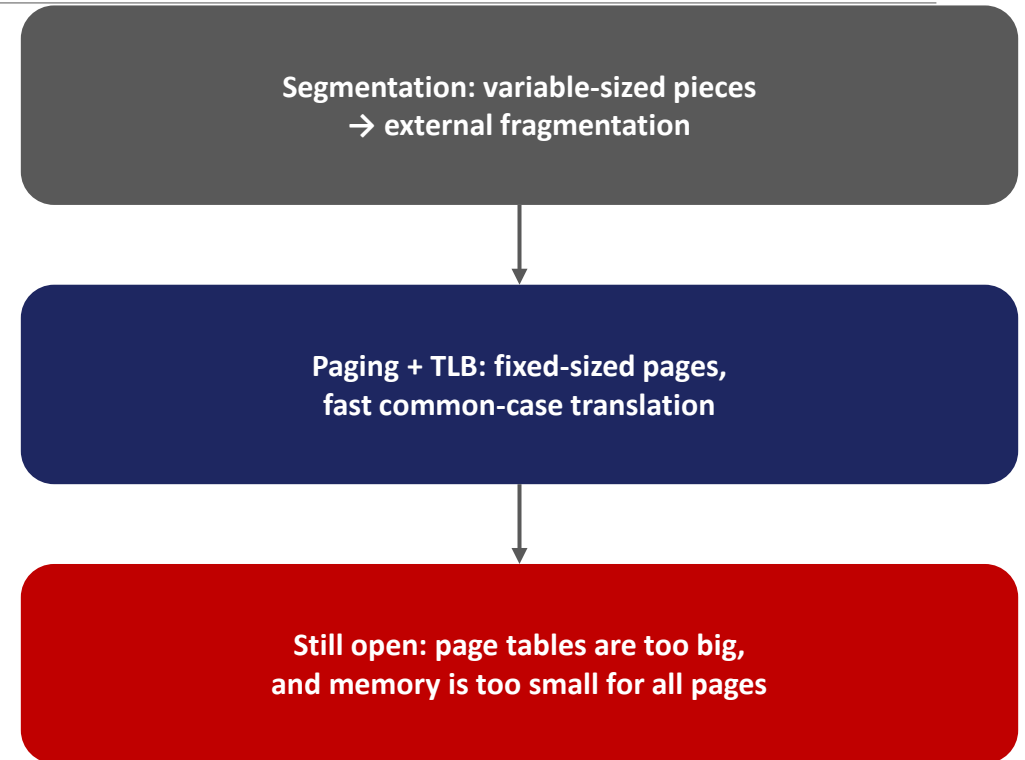
INSTRUCTOR: DEEPAK GANGADHARAN

Agenda

- **Part 1: Smaller Page Tables**
 - Bigger pages, hybrid paging + segmentation, multi-level tables, inverted tables
- **Part 2: Beyond Physical Memory — Mechanisms**
 - Swap space, the present bit, page faults, the page-fault control flow
- **Part 3: Beyond Physical Memory — Policies**
 - Optimal, FIFO, Random, LRU, Clock, dirty pages, thrashing

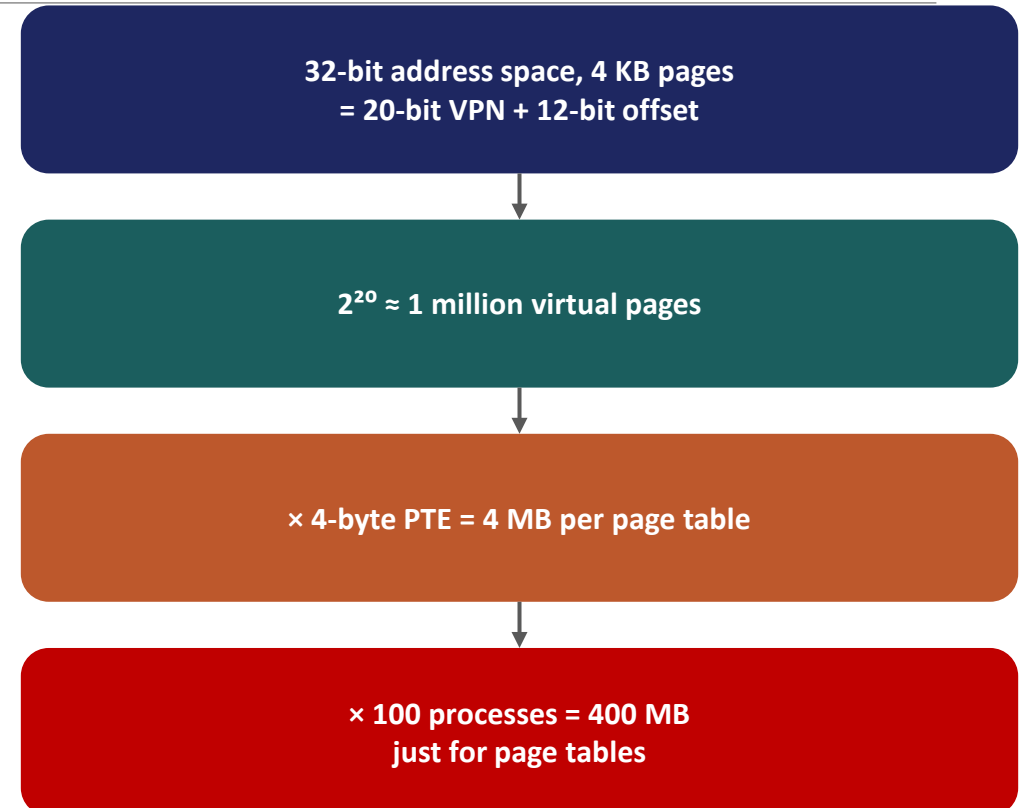
Recap: Where We Are

- Segmentation uses variable-sized pieces → external fragmentation
- Paging uses fixed-sized pages and page frames → no external fragmentation
- The TLB caches translations so most accesses skip the page table
- **Two problems remain**
 - Page tables are too big and consume too much memory
 - We assumed all pages of all processes fit in physical memory



The Problem: Linear Page Tables Are Big

- A linear page table is a simple array of PTEs indexed by VPN
- 32-bit address space with 4 KB (2^{12}) pages → about a million virtual pages
- With 4-byte PTEs the table is 4 MB in size
- There is usually one page table per process
- **100 active processes → hundreds of MB just for page tables**



The Crux: How to Make Page Tables Smaller?

- Simple array-based (linear) page tables take far too much memory
- **How can we make page tables smaller? What are the key ideas?**
- What inefficiencies arise from the new data structures?
- There are many techniques; we will look at five
- Each one trades something away: time, flexibility or complexity

1 Bigger pages

2 Hybrid: paging + segments

3 Multi-level page tables

4 Inverted page tables

5 Swap page tables to disk

Simple Fix: Use Bigger Pages

- Same 32-bit address space, but 16 KB pages instead of 4 KB
- VPN shrinks to 18 bits; offset grows to 14 bits
- 2^{18} entries $\times$ 4-byte PTE = 1 MB per page table
- A factor of four smaller — exactly mirroring the 4 $\times$ larger page
- Simple, but it does not solve the problem

Page-table size for a 32-bit address space

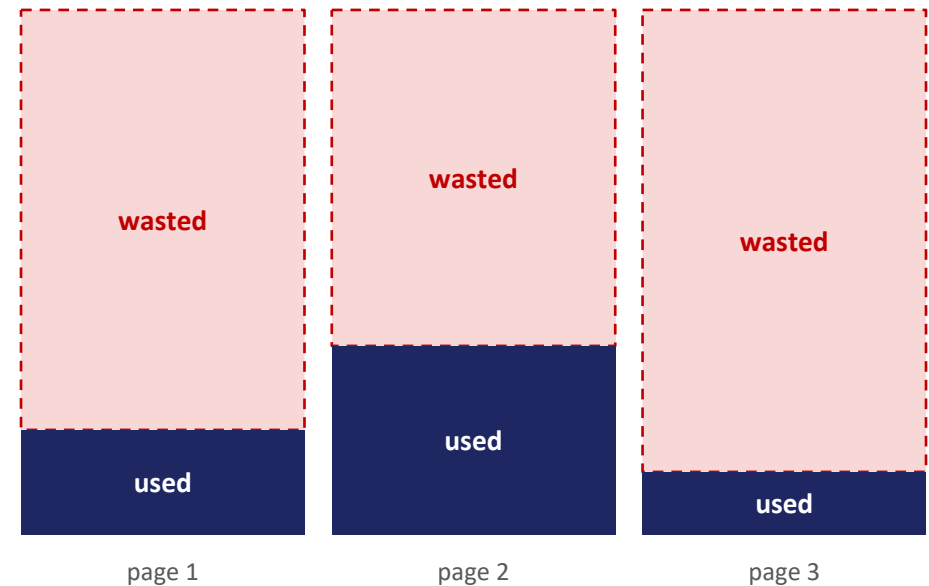
4 KB pages: 20-bit VPN
 2^{20} entries $\rightarrow$ 4 MB

16 KB pages: 18-bit VPN
 2^{18} entries $\rightarrow$ 1 MB

4 $\times$ bigger pages $\rightarrow$ 4 $\times$ smaller table
(the reduction mirrors the page-size increase)

Why Not Just Use Huge Pages?

- Big pages waste memory inside each page — internal fragmentation
- Applications allocate whole pages but use only bits and pieces of each
- Memory quickly fills with overly-large, mostly empty pages
- So systems use small pages: 4 KB (x86) or 8 KB (SPARCv9)
- **Aside: many CPUs also support multiple page sizes (e.g., 4 MB)**
 - Mainly to reduce TLB pressure, not to save page-table space



**Big pages → waste INSIDE each page
(internal fragmentation)**

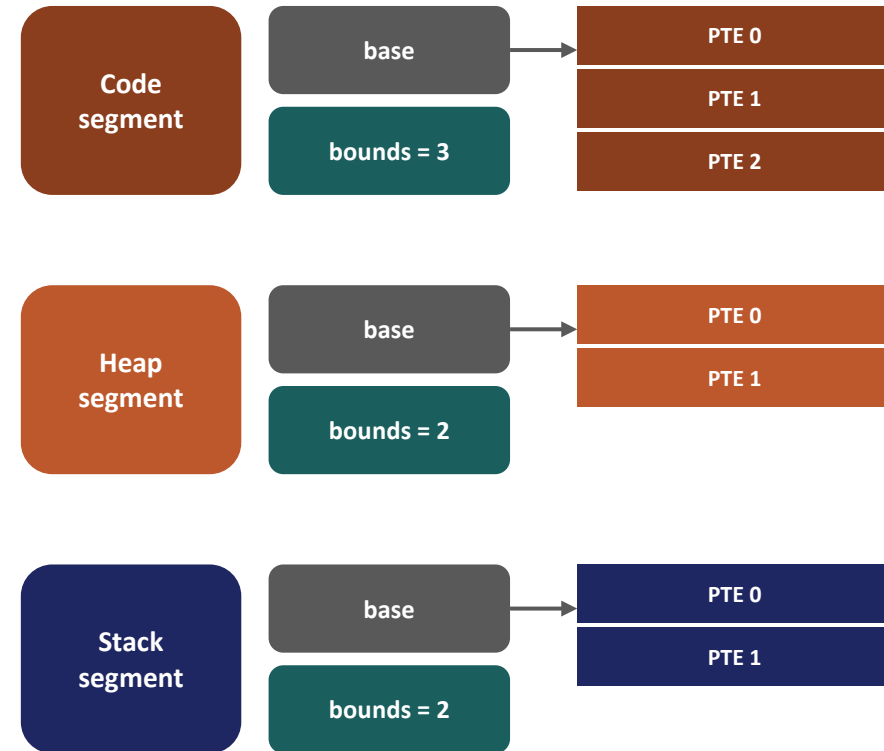
Hybrid: Paging + Segmentation

- When two reasonable approaches exist, examine a hybrid of the two
- Idea from Multics: combine paging and segmentation to shrink page tables
- Look at a linear table for a sparse address space: mostly invalid entries
- Tiny example: 16 KB address space, 1 KB pages, 16 entries
- Only 1 code page, 1 heap page and 2 stack pages are valid

Address space	Linear page table
VPN 0 code	PFN 10 · valid
VPN 1	invalid (wasted entry)
VPN 2	invalid (wasted entry)
VPN 3	invalid (wasted entry)
VPN 4 heap	PFN 23 · valid
VPN 5	invalid (wasted entry)
VPN 6	invalid (wasted entry)
VPN 7	invalid (wasted entry)
VPN 8	invalid (wasted entry)
VPN 9	invalid (wasted entry)
VPN 10	invalid (wasted entry)
VPN 11	invalid (wasted entry)
VPN 12	invalid (wasted entry)
VPN 13	invalid (wasted entry)
VPN 14 stack	PFN 28 · valid
VPN 15 stack	PFN 4 · valid

Hybrid: One Page Table per Segment

- Instead of one page table for the whole space, keep one per logical segment
- Code, heap and stack each get their own (much smaller) page table
- MMU keeps a base/bounds pair per segment, as in segmentation
- **Base register now holds the physical address of the segment's page table**
- **Bounds register holds how many pages are valid (end of the page table)**
- Context switch: these registers must be switched too



base → page table of the segment; bounds = # valid pages

Hybrid: Address Format and Translation

- 32-bit address, 4 KB pages, address space split into four segments
- Top two bits select the segment: 01 code, 10 heap, 11 stack, 00 unused
- Below the segment bits: a 18-bit VPN and a 12-bit offset
- On a TLB miss (hardware-managed): pick Base[SN] and add $\text{VPN} \times \text{sizeof(PTE)}$
- Virtually identical to linear tables — but with three base registers



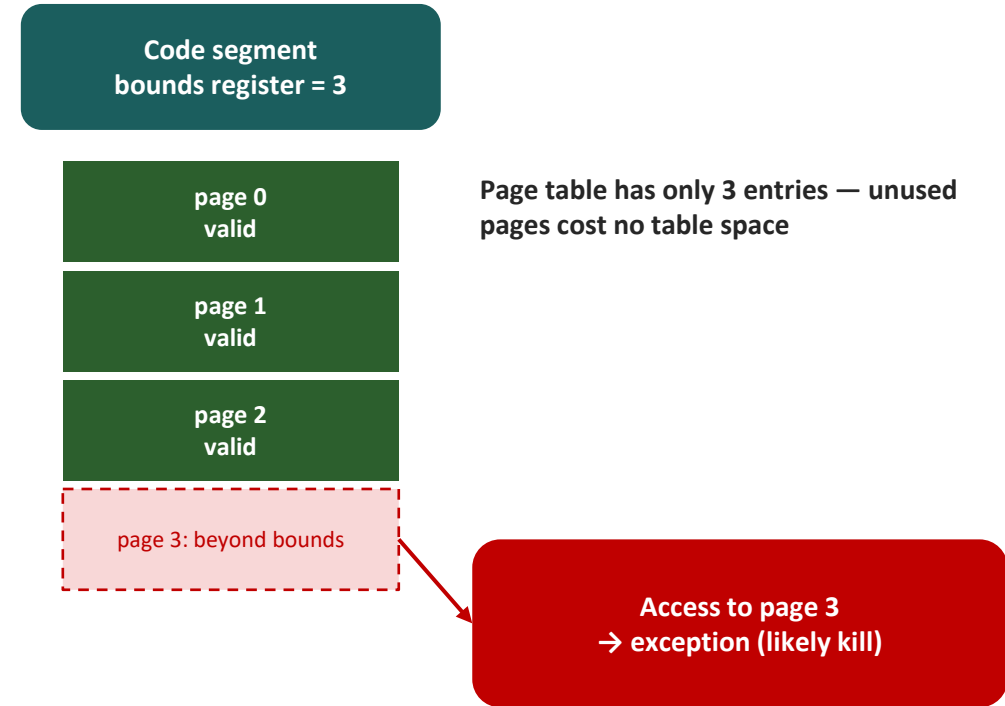
01 = code, 10 = heap, 11 = stack, 00 = unused

TLB miss (hardware-managed)

```
SN = (VirtualAddress & SEG_MASK) >> SN_SHIFT
VPN = (VirtualAddress & VPN_MASK) >> VPN_SHIFT
AddressOfPTE = Base[SN] + (VPN * sizeof(PTE))
```

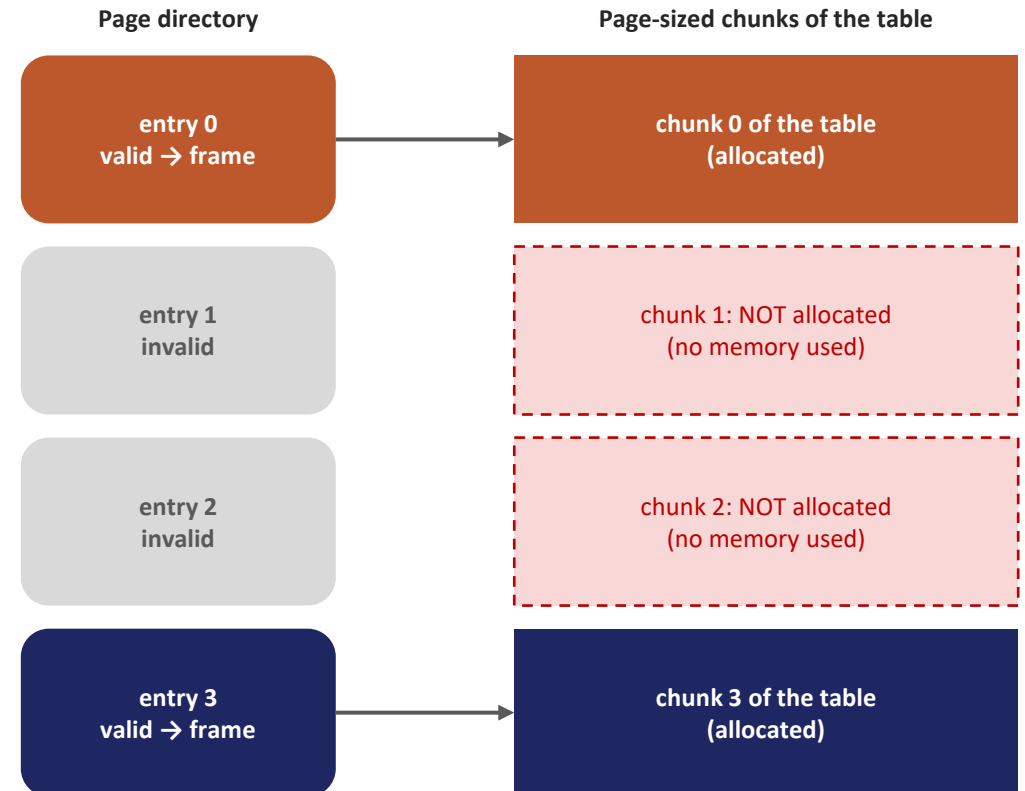
Hybrid: Bounds Registers and Drawbacks

- Each bounds register holds the number of valid pages in its segment
- Code using pages 0, 1, 2 → page table has 3 entries, bounds = 3
- Access beyond the bounds raises an exception
- Unallocated pages between heap and stack no longer cost table space
- But: it still needs segmentation, which assumes a usage pattern**
- And: page tables now vary in size → external fragmentation returns**



Multi-Level Page Tables: The Idea

- Goal: get rid of the invalid regions without relying on segmentation
- Step 1: chop the page table into page-sized pieces
- Step 2: if a whole page of PTEs is invalid, don't allocate it at all
- Step 3: track which pieces exist with a new structure: the page directory
- The linear table becomes a tree; many modern systems (e.g., x86) use it



The Page Directory and Its Entries

- A two-level directory has one entry (PDE) per page of the page table
- A PDE has (at minimum) a valid bit and a page frame number (PFN)
- **PDE valid $\Rightarrow$ at least one PTE on the pointed-to page is valid**
- PDE invalid $\Rightarrow$ the rest of the entry is undefined
- The valid bit therefore means something slightly different than in a PTE

Page-table entry (PTE)



Page-directory entry (PDE)

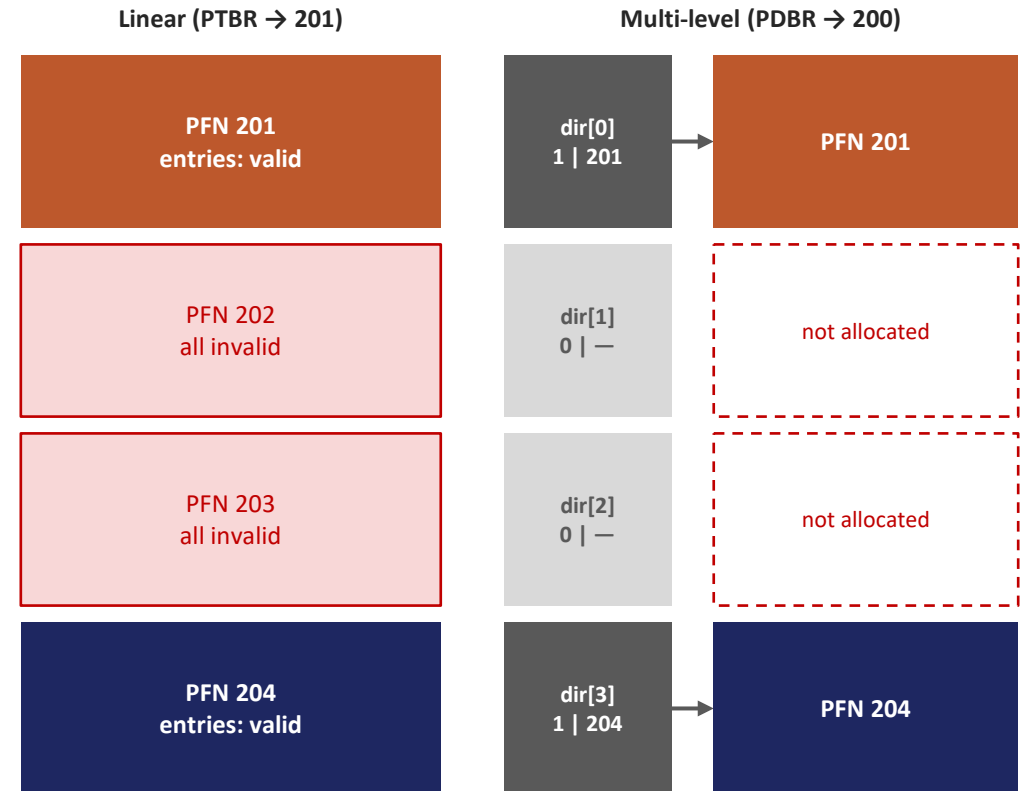


PDE valid $\Rightarrow$ at least one PTE on the pointed-to page is valid

PDE invalid $\Rightarrow$ the rest of the entry is undefined;
the whole page of PTEs is absent

Linear vs. Multi-Level Page Table

- Left: a linear table whose middle pages are entirely invalid but still allocated
- Right: the page directory marks only the first and last pages as valid
- Only those two pages of the table reside in memory
- The freed frames can be used for something else
- The directory tracks which pages of the table are allocated



Multi-Level: Benefits and Costs

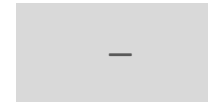
■ Benefits

- Page-table space is proportional to the address space actually used
- Each piece of the table fits in one page — easy to allocate and grow
- No large contiguous chunk is needed: the directory lets pieces live anywhere

■ Costs

- TLB miss needs two loads (PDE, then PTE) instead of one
- More complex lookup — a classic time-space trade-off

TLB hit



no page-table access (identical for both)

TLB miss — linear table



1 extra load: the PTE

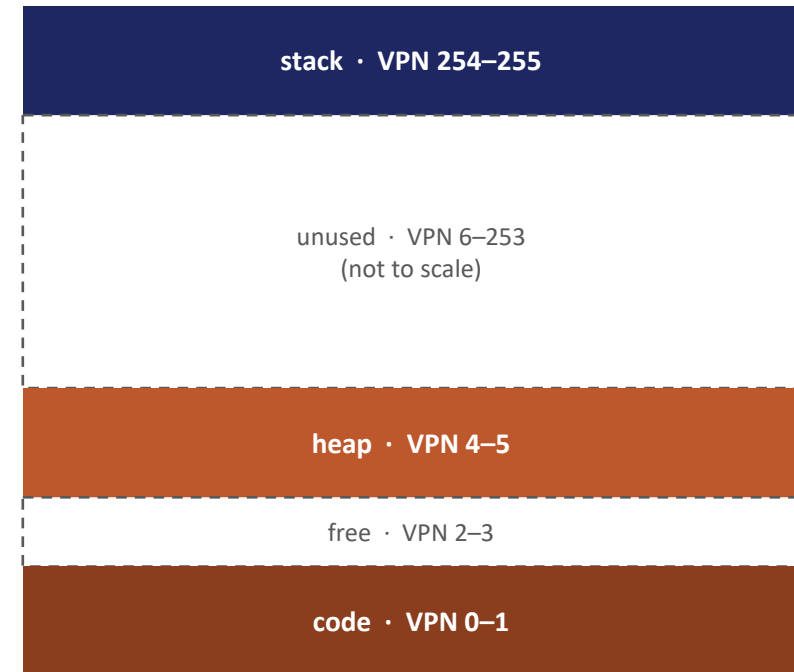
TLB miss — two-level table



2 extra loads: the PDE, then the PTE

Example: A 16 KB Address Space

- Address space of 16 KB with 64-byte pages
- 14-bit virtual address: 8-bit VPN + 6-bit offset
- A linear table would have $2^8 = 256$ entries
- VPNs 0–1 hold code; VPNs 4–5 hold the heap
- VPNs 254–255 hold the stack; everything else is unused



14-bit virtual address = 8-bit VPN + 6-bit offset
linear table: $2^8 = 256$ entries

Example: Chopping the Table into Pages

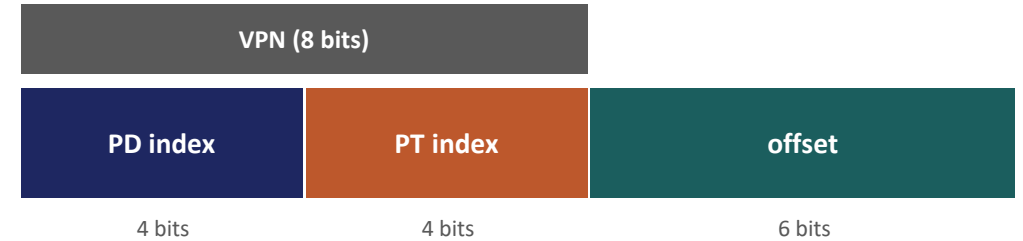
- Start from the full linear table: $256 \text{ entries} \times 4 \text{ bytes} = 1 \text{ KB}$
- With 64-byte pages, the 1 KB table divides into 16 pages
- Each page of the table holds 16 PTEs
- Only page 0 (VPNs 0–15) and page 15 (VPNs 240–255) contain valid entries
- The other 14 pages of the table are entirely invalid

page 0 VPN 0–15	page 1 all invalid	page 2 all invalid	page 3 all invalid
page 4 all invalid	page 5 all invalid	page 6 all invalid	page 7 all invalid
page 8 all invalid	page 9 all invalid	page 10 all invalid	page 11 all invalid
page 12 all invalid	page 13 all invalid	page 14 all invalid	page 15 VPN 240–255

$256 \text{ PTEs} \times 4 \text{ B} = 1 \text{ KB} = 16 \text{ pages of } 64 \text{ B}$
(each page of the table holds 16 PTEs)

Example: Indexing the Directory and the Table

- Directory has 16 entries → 4 bits of the VPN index it (the top 4 bits)
- $PDEAddr = PageDirBase + PDIndex \times sizeof(PDE)$
- A valid PDE gives the PFN of a page of the page table
- The remaining 4 VPN bits index that page: the page-table index
- $PTEAddr = (PDE.PFN \ll SHIFT) + PTIndex \times sizeof(PTE)$



two index calculations

```
PDEAddr = PageDirBase + (PDIndex * sizeof(PDE))  
PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
```

Directory needs 1 entry per page of the table: 16 entries → 4 index bits

Example: Directory and Table Contents

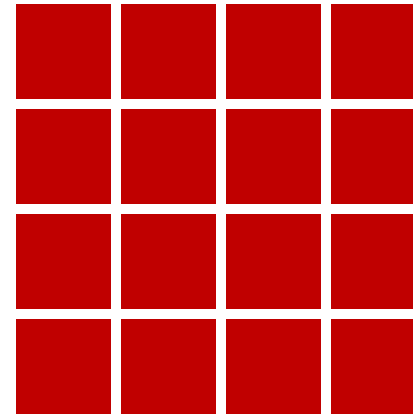
- Directory entry 0 → PFN 100 (valid); entry 15 → PFN 101 (valid)
- Entries 1–14 are invalid: no page of the table exists for them
- PFN 100 holds PTEs for VPNs 0–15: VPNs 0, 1 (code) and 4, 5 (heap) are valid
- PFN 101 holds PTEs for VPNs 240–255: VPNs 254, 255 (stack) are valid
- All other PTEs in those two pages are marked invalid

Page directory (16 entries)	Page of PT @ PFN 100	Page of PT @ PFN 101
0: PFN 100, valid	VPN 0: 10 r-x	
	VPN 1: 23 r-x	
	VPN 2–3: invalid	
	VPN 4: 80 rw-	
	VPN 5: 59 rw-	
1–14: invalid	VPN 6–15: invalid	VPN 240–253: invalid
15: PFN 101, valid		VPN 254: 55 rw-
		VPN 255: 45 rw-

Example: Space Savings

- Linear table: 16 pages of memory for the table
- Two-level table: 1 page for the directory + 2 pages of the table = 3 pages
- About a five-fold reduction in this tiny example
- Savings for 32-bit or 64-bit address spaces are far greater
- Unused regions cost essentially nothing

Linear table: 16 pages



Multi-level: 3 pages

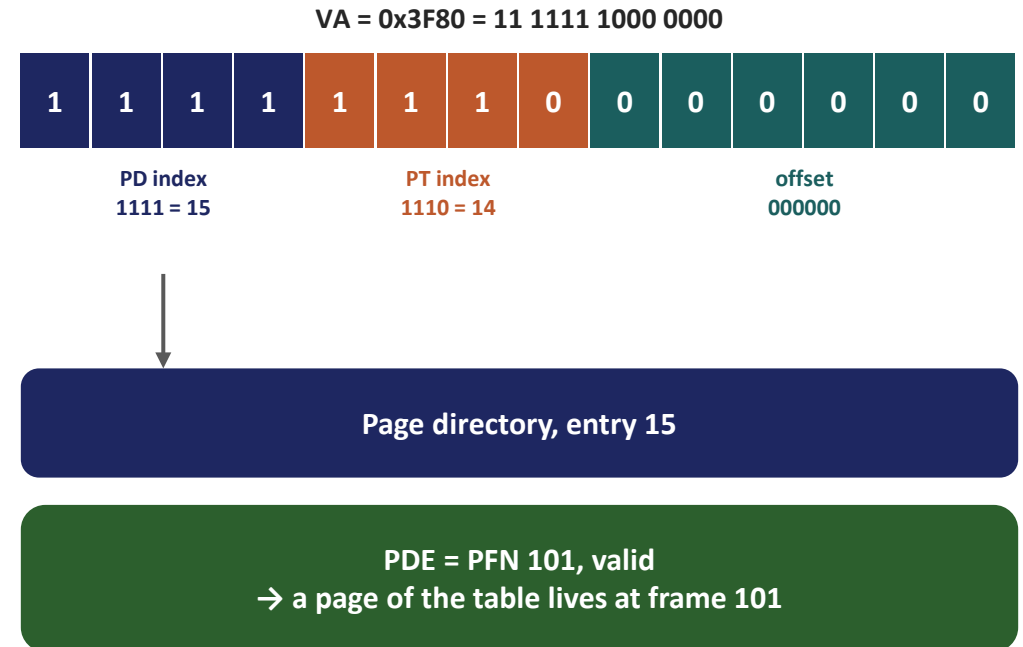


1 directory page
+ 2 pages of the table

Same address space, ~5× less memory for translation.
For 32- or 64-bit address spaces the savings are far larger.

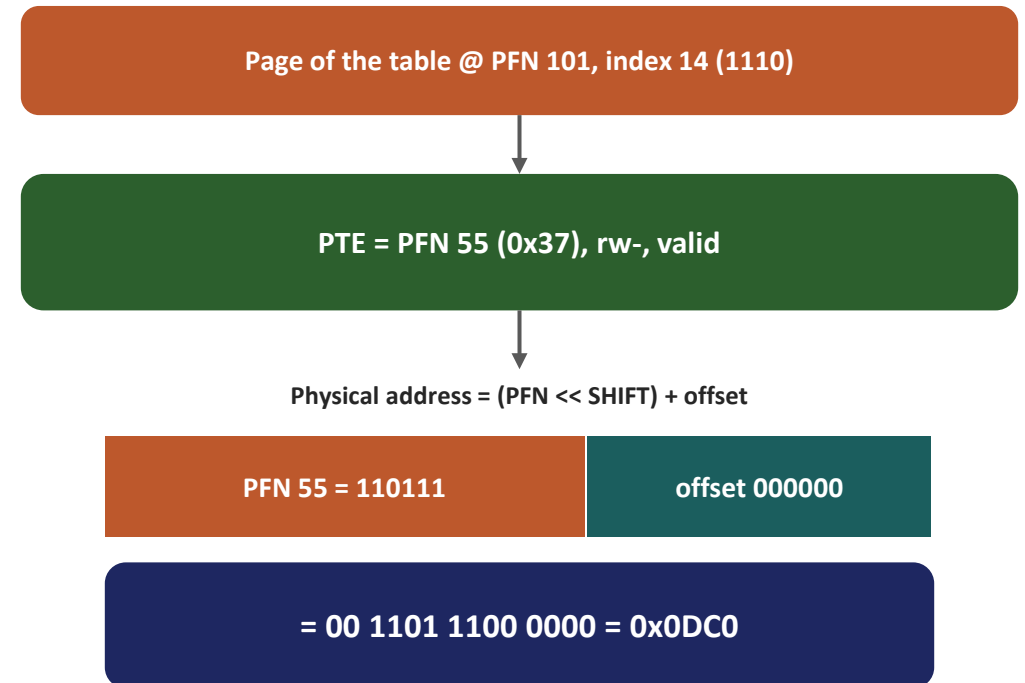
Translating 0x3F80: The Directory Lookup

- Address 0x3F80 refers to the 0th byte of VPN 254
- Binary: 11 1111 1000 0000 → VPN = 1111 1110, offset = 000000
- Top 4 VPN bits (1111) pick page-directory entry 15
- That PDE is valid and points to the page of the table at PFN 101
- Next: use the other 4 VPN bits to index into that page



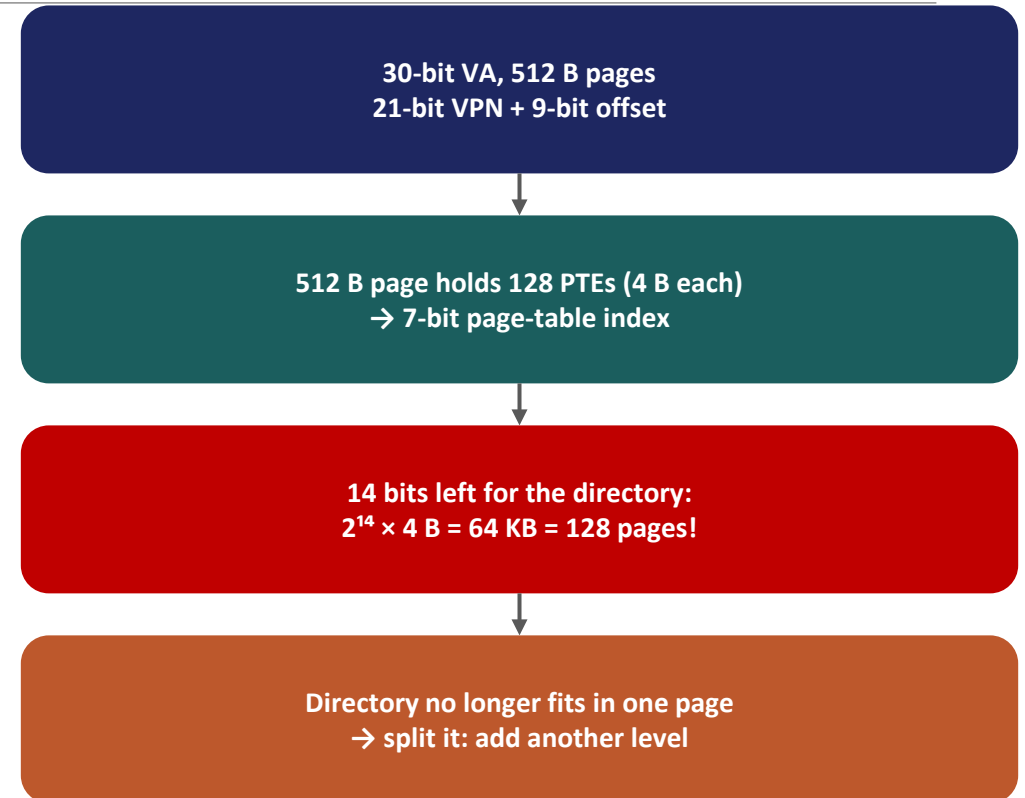
Translating 0x3F80: The Page-Table Lookup

- The next 4 VPN bits (1110 = 14) index the page of the table at PFN 101
- Entry 14 is the next-to-last PTE and is valid
- It maps virtual page 254 to physical frame 55 (0x37)
- Concatenate PFN 55 with the 6-bit offset 000000
- $\text{PhysAddr} = (\text{PFN} \ll \text{SHIFT}) + \text{offset} = 00\ 1101\ 1100\ 0000 = 0x0DC0$



More Than Two Levels: The Problem

- Example: 30-bit virtual address, small 512-byte pages
- 21-bit VPN + 9-bit offset
- A 512-byte page holds 128 4-byte PTEs → 7-bit page-table index
- That leaves 14 bits for the directory: 2^{14} entries $\times$ 4 B = 64 KB
- **The directory spans 128 pages: it no longer fits in one page**

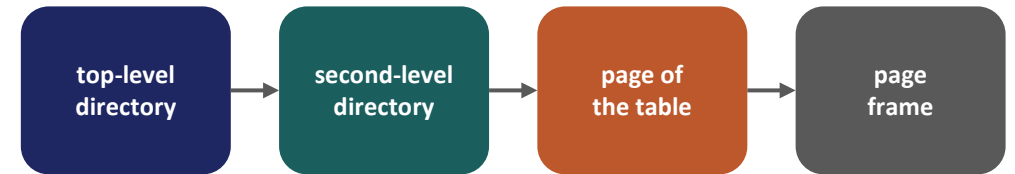


Three-Level Address Split

- Split the 14 directory bits in two: PD index 0 and PD index 1 (7 bits each)
- Top bits index the top-level directory
- Its PFN locates a page of the second-level directory
- PD index 1 indexes that page; its PFN locates a page of the page table
- The page-table index then gives the final PTE



7 + 7 + 7 index bits, 9 offset bits (30 bits total)



Each index selects an entry; each entry's PFN locates the next level, until the final PFN is found

Multi-Level Translation: Control Flow

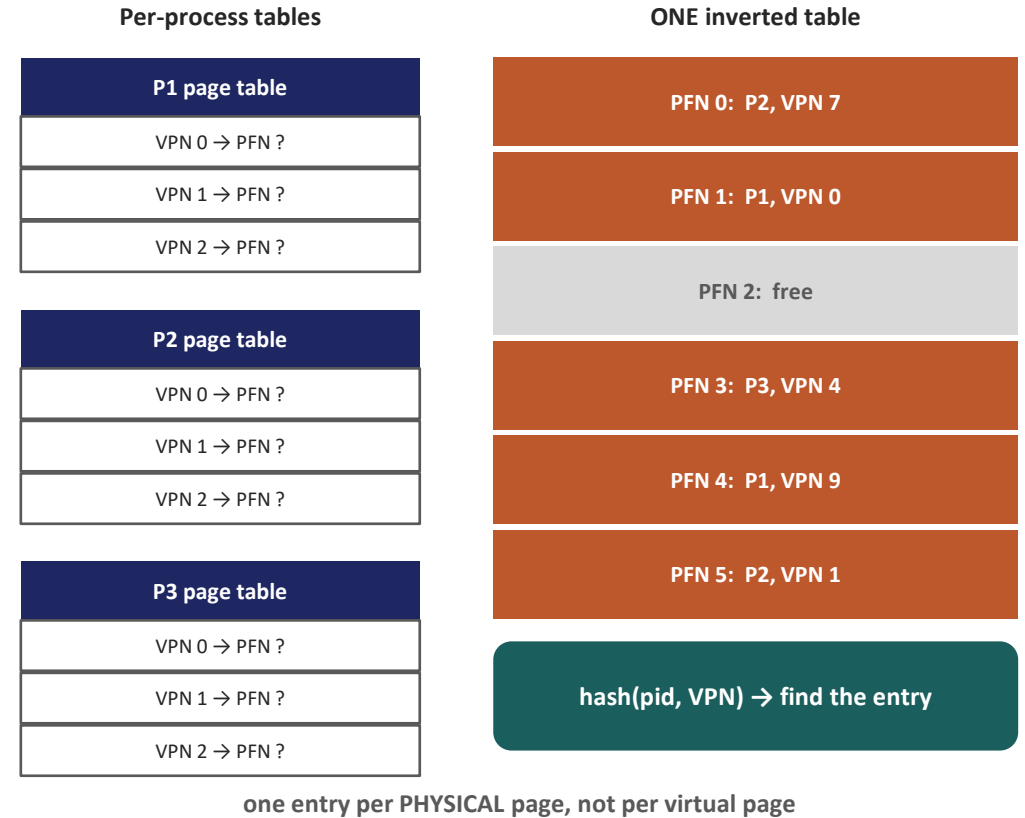
- Remember the TLB: check it before any multi-level work
- TLB hit: form the physical address directly, no page-table access
- TLB miss: read the PDE, check valid, then read the PTE
- Invalid PDE or PTE → raise a fault; permission failure → protection fault
- Cost of a miss: two extra memory accesses (for a two-level table)

hardware-managed TLB, two-level table

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN)
if (Success == True) // TLB Hit
    if (CanAccess(TlbEntry.ProtectBits) == True)
        Offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
        Register = AccessMemory(PhysAddr)
    else
        RaiseException(PROTECTION_FAULT)
else // TLB Miss
    PDIndex = (VPN & PD_MASK) >> PD_SHIFT
    PDEAddr = PDBR + (PDIndex * sizeof(PDE))
    PDE = AccessMemory(PDEAddr)
    if (PDE.Valid == False)
        RaiseException(SEGMENTATION_FAULT)
    else
        PTIndex = (VPN & PT_MASK) >> PT_SHIFT
        PTEAddr = (PDE.PFN << SHIFT) + (PTIndex * sizeof(PTE))
        PTE = AccessMemory(PTEAddr)
        if (PTE.Valid == False)
            RaiseException(SEGMENTATION_FAULT)
        else if (CanAccess(PTE.ProtectBits) == False)
            RaiseException(PROTECTION_FAULT)
        else
            TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
            RetryInstruction()
```

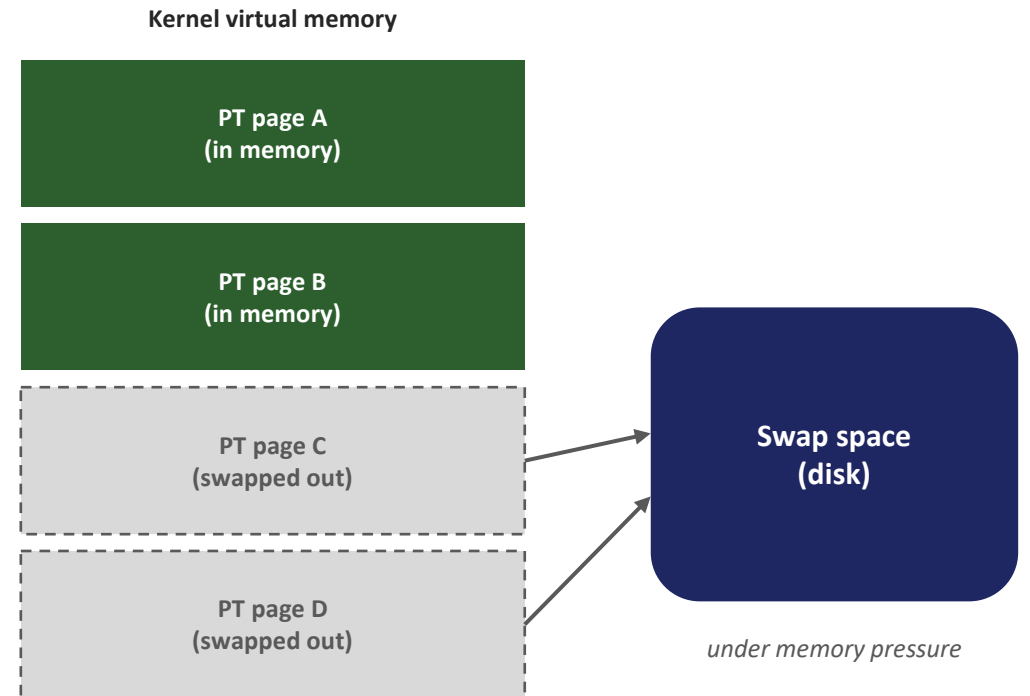
Inverted Page Tables

- An even more extreme space saving
- ONE page table for the whole system, not one per process
- One entry per PHYSICAL page: which process and which virtual page use it
- Finding an entry means searching — a hash table is built on top to speed it up
- PowerPC is an example; page tables are just data structures



Swapping Page Tables to Disk

- So far we assumed page tables sit in kernel-owned physical memory
- Even with smaller tables, they may still be too big to fit at once
- Some systems place page tables in kernel virtual memory
- Then parts of the page tables can be swapped to disk under memory pressure
- This relaxes our last assumption about where page tables live



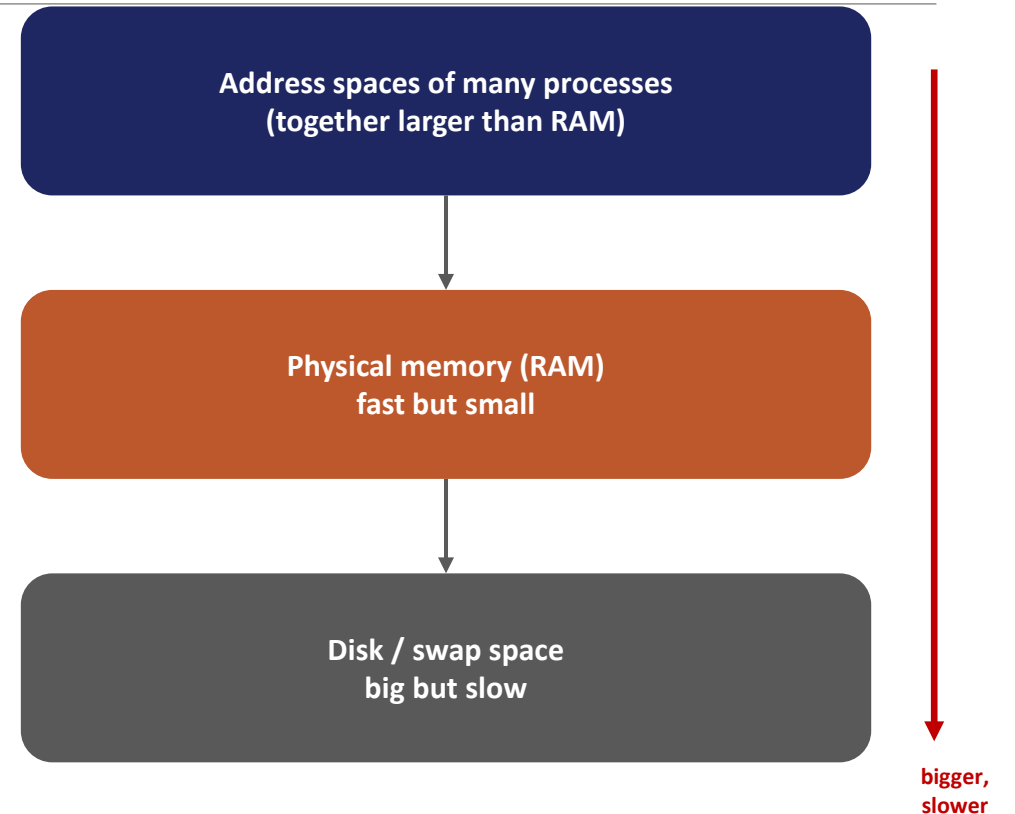
Summary: Shrinking Page Tables

- Real page tables are more complex data structures than simple arrays
- The trade-off is time versus space: bigger tables make TLB misses faster
- Memory-constrained systems favor small structures
- Systems with ample memory and many active pages may prefer bigger, faster tables
- With software-managed TLBs, the OS may use any structure it likes

Approach	Saves space by	Costs
Bigger pages	fewer entries	internal fragmentation
Hybrid (paging + segments)	one small table per segment	needs segmentation; external fragmentation
Multi-level	allocate only the used pieces	extra loads on a TLB miss; complexity
Inverted	one entry per physical page	search needed (hash table)

Beyond Physical Memory: The Crux

- So far every address space of every process fit entirely in memory
- We now want many concurrent, LARGE address spaces
- This needs another level in the memory hierarchy
- It must be bigger than memory → therefore slower (typically a hard disk)
- **Crux:** how can the OS use a larger, slower device to transparently provide the illusion of a large virtual address space?



Why Support Large Address Spaces?

- Convenience and ease of use: no worrying about whether data fits in memory
- Just write the program naturally, allocating memory as needed
- Contrast: older systems used memory overlays
 - Programmers manually moved code or data in and out of memory
- Multiprogramming plus ease of use almost demands swapping pages out
- All modern VM systems use more memory than is physically available

Overlays (older systems)

Programmer must manually load code or data into memory BEFORE every function call or data access — and unload it again

Virtual memory (modern systems)

OS + hardware move pages between RAM and disk transparently — write the program naturally, allocate as you need

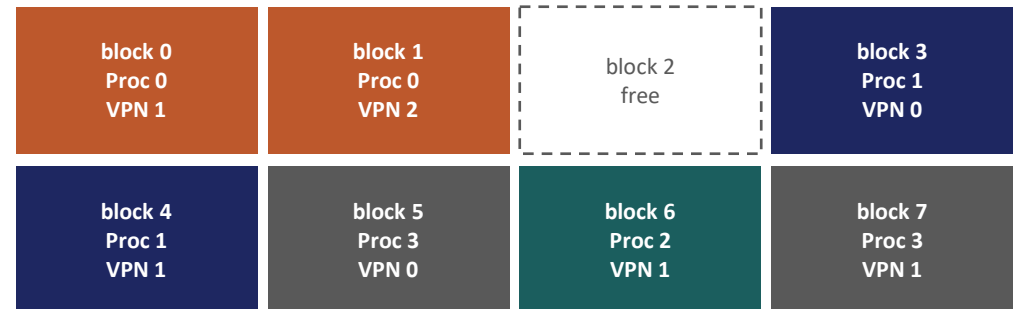
Swap Space

- Reserve space on disk for moving pages back and forth: swap space
- Pages are swapped out of memory to it, and swapped in from it
- The OS reads and writes swap in page-sized units
- The OS must remember the disk address of each swapped page
- Swap size bounds the maximum number of pages in use at a time

Physical memory (4 frames)



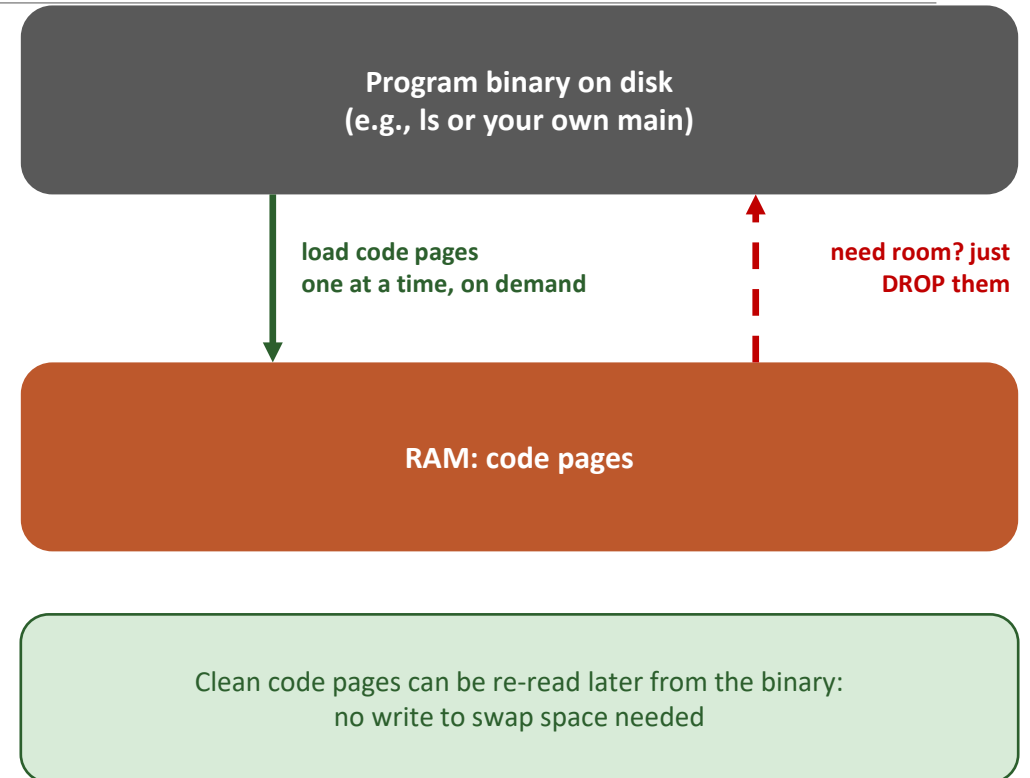
Swap space on disk (8 blocks)



Proc 3 has all pages on disk (not running).
Swap lets the system pretend memory is larger.

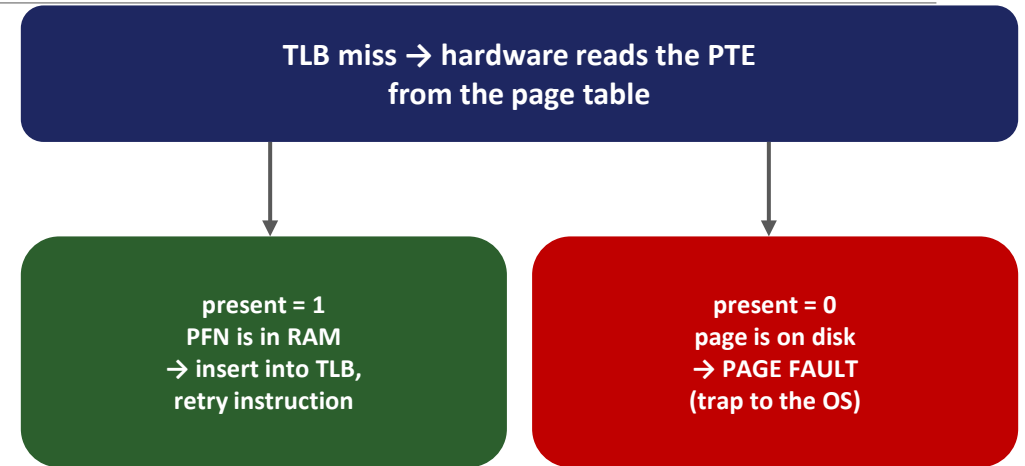
Swap Space Isn't the Only Disk Location

- Program binaries (e.g., ls or your own program) live on disk
- Code pages are loaded into memory — all at once or, in modern systems, one page at a time
- To free memory, the OS can simply reuse the frames of code pages
- It can safely do so because it can swap them in again from the binary
- Such clean pages never need to be written to swap



The Present Bit

- Assume a hardware-managed TLB; recall the usual flow
- Hardware extracts the VPN and checks the TLB; a hit is fast
- On a miss, hardware reads the PTE using the page-table base register
- New: a present bit in each PTE says whether the page is in memory**
- present = 1 → install the translation in the TLB and retry
- present = 0 → the page is on disk → page fault



The present bit is the new PTE bit that makes swapping possible.

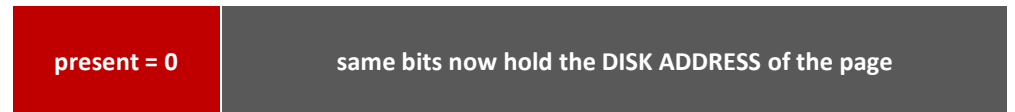
PTE: Present Bit and Disk Address

- Present bit set: the PFN field says which frame holds the page
- Present bit clear: the page is on disk
- Where on disk? The PTE is a natural place to store that address
- The OS reuses bits normally used for the PFN to hold a disk address
- On a fault the OS reads the PTE to find where to fetch the page

Page is in memory



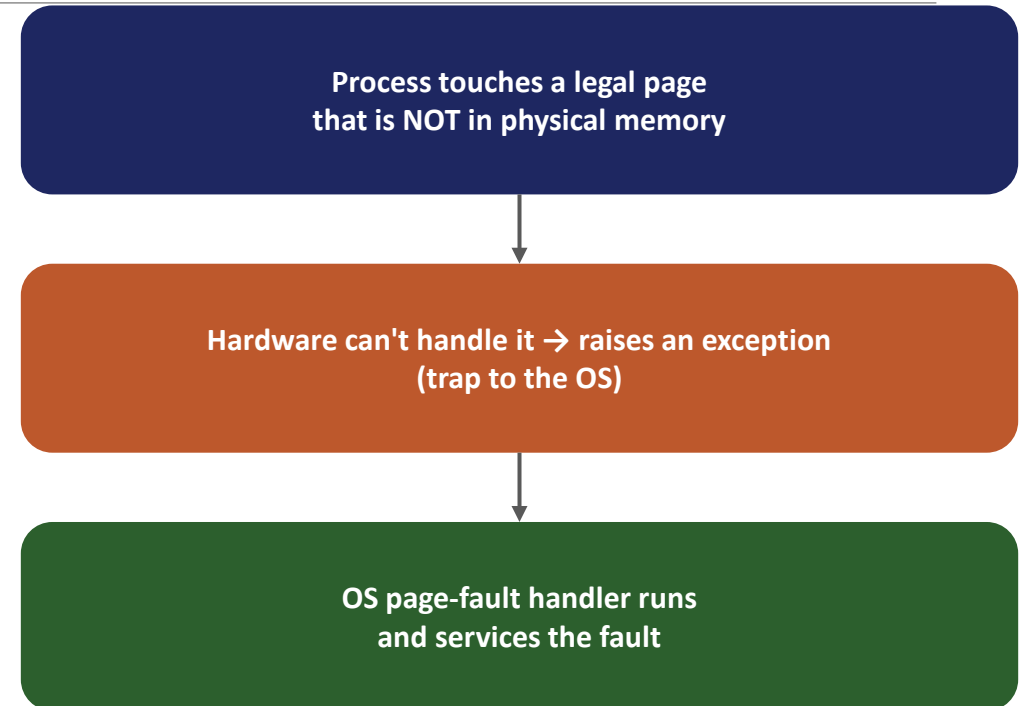
Page is swapped out



The PTE is a natural place to remember where on disk the page lives — the OS reuses the PFN bits for that address

The Page Fault

- Accessing a page not in physical memory is a page fault
- The OS runs a page-fault handler to service it
- Terminology is confusing: a “fault” here is a perfectly legal access
 - It could really be called a page miss
- Hardware can't handle it, so it raises an exception — same as an illegal access
- That shared mechanism is probably why it is called a “fault”



Who Handles a Page Fault?

- Recall TLB misses: hardware-managed or software-managed
- For page faults, virtually all systems use the OS, even with hardware TLBs
- **Reason 1: disk is so slow that software overhead is negligible**
- **Reason 2: hardware would need to understand swap space and disk I/O**
- For performance and simplicity, the OS handles page faults

Why not handle page faults in hardware?

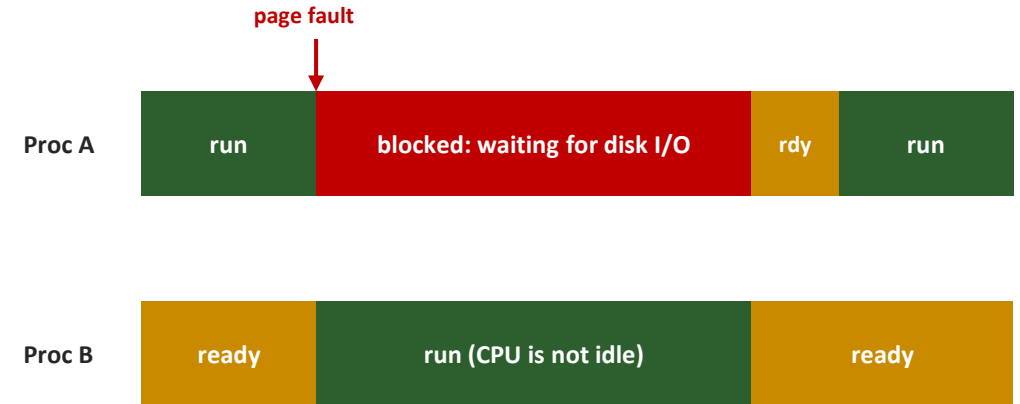
1. The disk is so slow that running OS software adds almost nothing to the total time
2. Hardware would have to understand swap space, issue disk I/Os, and many other details

So the OS handles it

Performance and simplicity: even hardware designers (who rarely trust the OS) agree.

Servicing a Page Fault

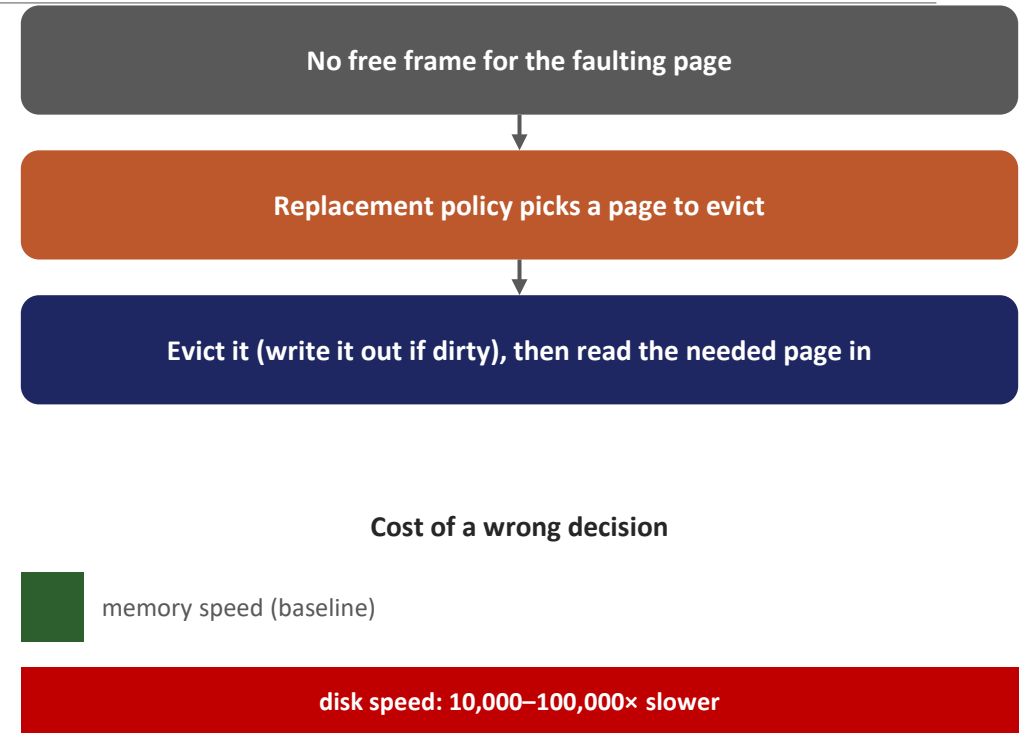
- The OS finds the disk address in the PTE and issues a read request
- While the I/O is in flight, the faulting process is blocked
- The OS is free to run other ready processes in the meantime
- When the I/O completes: mark the page present and record its PFN
- Retry: maybe a TLB miss (update the TLB), then a final retry hits



While A's page is being read, the OS runs B.
Overlapping I/O with computation is another way
multiprogramming uses hardware effectively.

What If Memory Is Full?

- We assumed there was a free frame to page the data into
- Memory may be full or close to it
- Then the OS must first page out one or more pages to make room
- Choosing the victim is the page-replacement policy
- **A bad choice can make a program run at disk speed: 10,000–100,000× slower**



Page-Fault Control Flow: Hardware

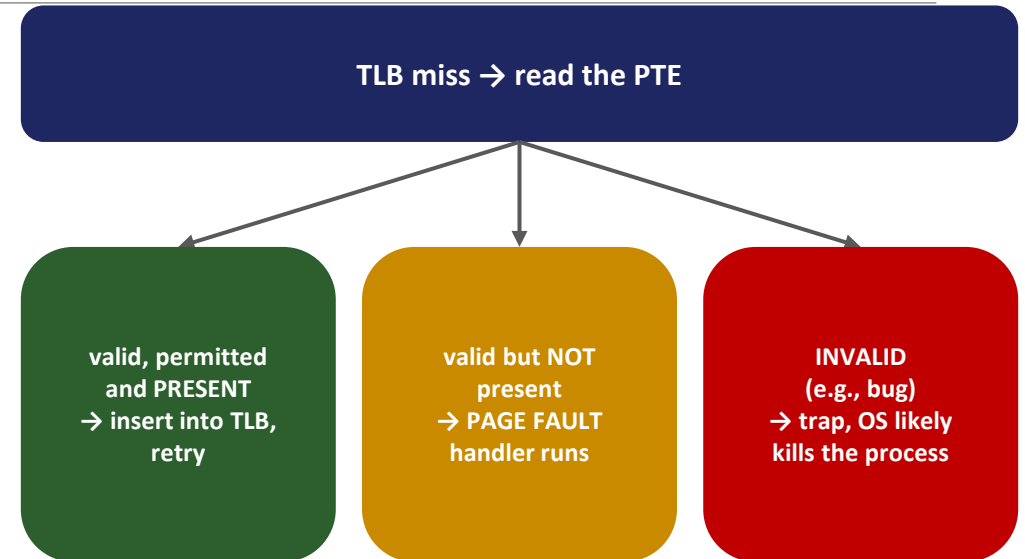
- The TLB check is unchanged: a hit forms the physical address
- On a miss, read the PTE and check the valid bit
- Then check protection bits
- **New: check the present bit**
 - present → insert into the TLB and retry
 - not present → raise PAGE_FAULT

what the hardware does on a memory reference

```
VPN = (VirtualAddress & VPN_MASK) >> SHIFT
(Success, TlbEntry) = TLB_Lookup(VPN)
if (Success == True) // TLB Hit
    if (CanAccess(TlbEntry.ProtectBits) == True)
        Offset = VirtualAddress & OFFSET_MASK
        PhysAddr = (TlbEntry.PFN << SHIFT) | Offset
        Register = AccessMemory(PhysAddr)
    else
        RaiseException(PROTECTION_FAULT)
else // TLB Miss
    PTEAddr = PTBR + (VPN * sizeof(PTE))
    PTE = AccessMemory(PTEAddr)
    if (PTE.Valid == False)
        RaiseException(SEGMENTATION_FAULT)
    else
        if (CanAccess(PTE.ProtectBits) == False)
            RaiseException(PROTECTION_FAULT)
        else if (PTE.Present == True)
            // assuming hardware-managed TLB
            TLB_Insert(VPN, PTE.PFN, PTE.ProtectBits)
            RetryInstruction()
        else if (PTE.Present == False)
            RaiseException(PAGE_FAULT)
```

Three Cases on a TLB Miss

- Case 1: the page is valid, permitted and present
 - The miss handler grabs the PFN, updates the TLB, retries — as before
- Case 2: the page is valid but not present
 - The page-fault handler must run; the access was legitimate
- Case 3: the access is to an invalid page (e.g., a program bug)
 - The OS trap handler runs, likely terminating the process



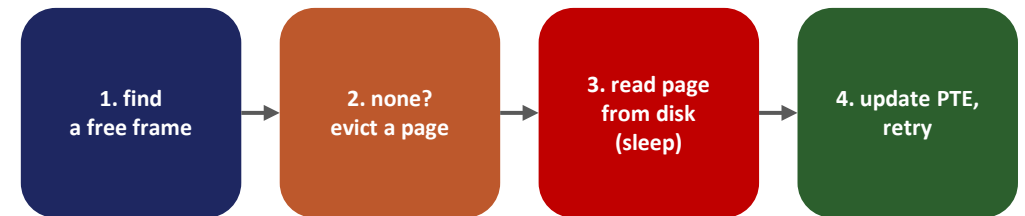
Three outcomes on a TLB miss.
Only the middle one involves the disk.

Page-Fault Control Flow: Software

- The OS must first find a physical frame for the incoming page
- If none is free, run the replacement algorithm to evict a page
- With a frame in hand, issue the disk read and wait
- When it completes, update the PTE: present = True and the new PFN
- Retry the instruction — TLB miss, then TLB hit

OS page-fault handler

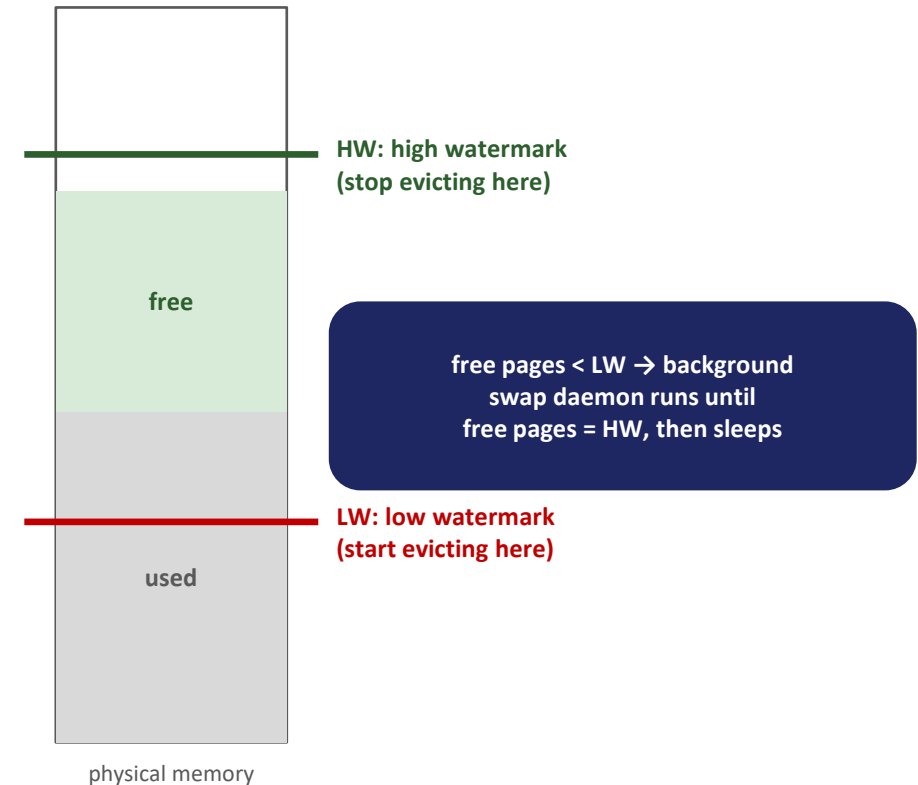
```
PFN = FindFreePhysicalPage()
if (PFN == -1)                // no free page found
    PFN = EvictPage()          // replacement algorithm
DiskRead(PTE.DiskAddr, PFN)    // sleep (wait for I/O)
PTE.present = True            // update page table:
PTE.PFN      = PFN            // (present/translation)
RetryInstruction()             // retry instruction
```



After the retry the access may still TLB-miss once, then hit — that is fine.

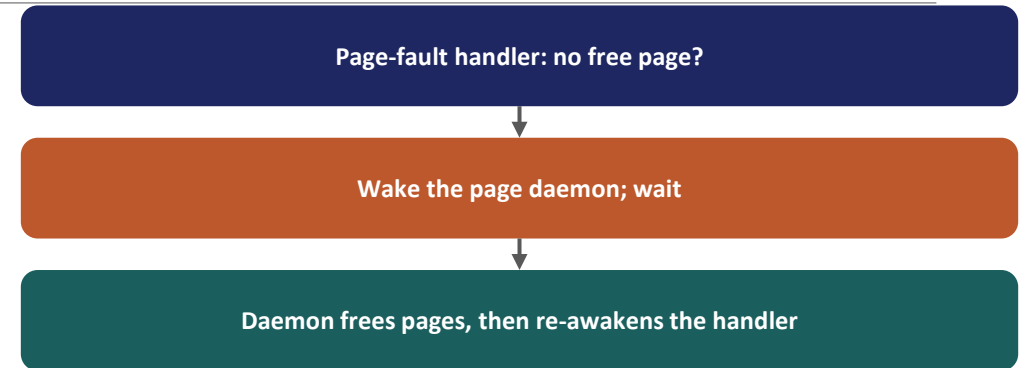
When Replacements Really Occur

- Waiting until memory is entirely full is unrealistic
- OSes proactively keep a small amount of memory free
- **High watermark (HW) and low watermark (LW)**
- Fewer than LW free pages → a background thread starts evicting
- It evicts until HW pages are free, then goes to sleep
- This thread is the swap daemon or page daemon



Background Work and Clustering

- **Tip: do work in the background**
 - Increases efficiency and allows grouping of operations
- By evicting many pages at once, new optimizations become possible
- Clustering: write a group of pages to swap in one operation
- This reduces disk seek and rotational overheads
- The fault handler asks the daemon for free pages instead of evicting itself



Clustering pending writes

one by one:

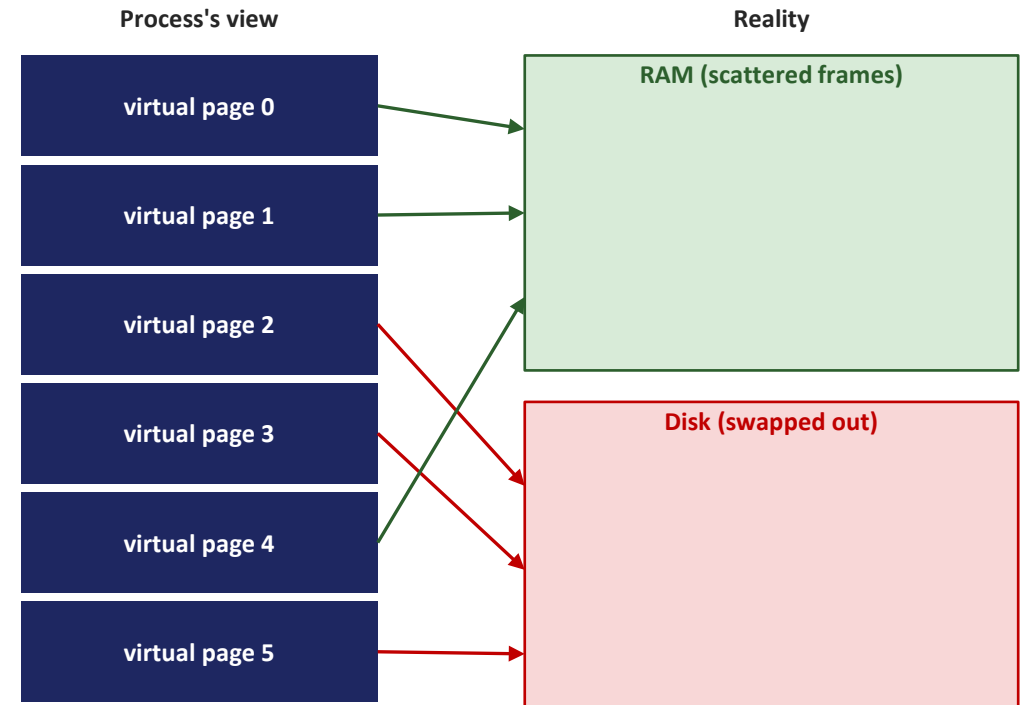


clustered:



Summary: Swapping Mechanisms

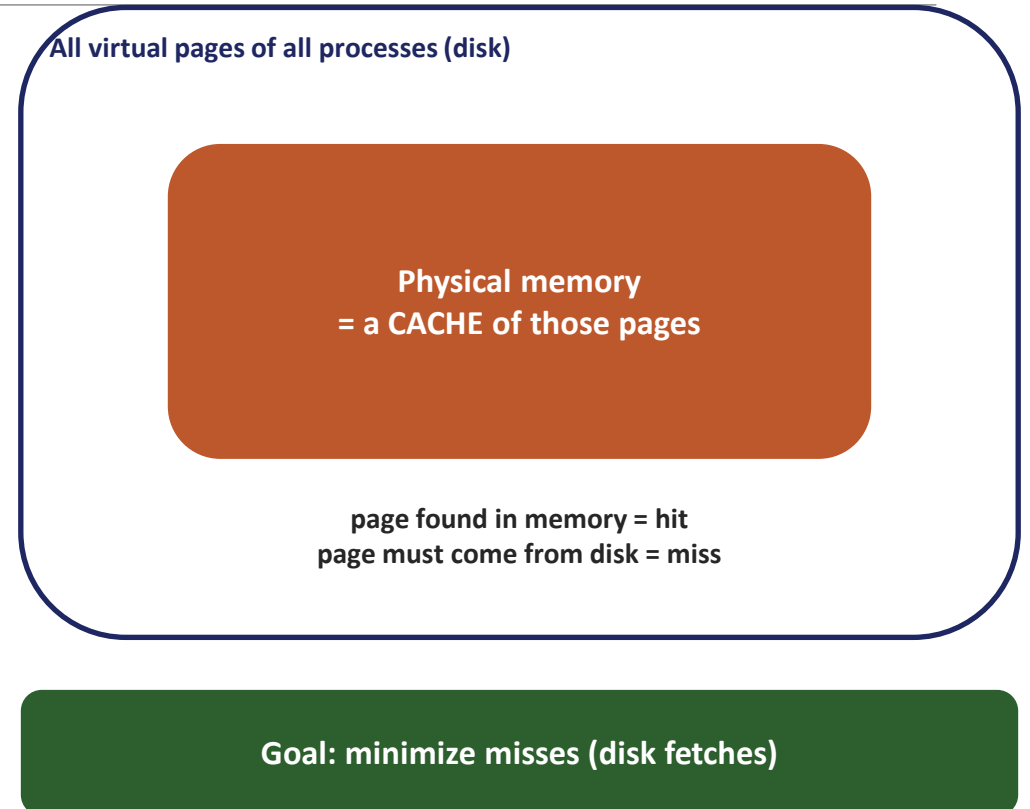
- To access more memory than exists, PTEs need a present bit
- A not-present page triggers the OS page-fault handler
- The handler fetches the page from disk, perhaps evicting another first
- **All of this is transparent to the process**
- Pages sit at arbitrary physical locations — or on disk
- **Worst case: one instruction can take many milliseconds**



The process still sees one private, contiguous space.

Which Page to Evict? The Crux

- With little free memory, the OS must page out to make room
- This memory pressure forces eviction of pages
- Deciding which page to evict is the replacement policy
- Memory holds a subset of all pages → it is a CACHE for virtual-memory pages
- **Goal: minimize misses (fetches from disk), i.e., maximize hits**



Average Memory Access Time (AMAT)

- Knowing hits and misses lets us compute AMAT
- $AMAT = T_M + P_{miss} \times T_D$
- T_M : cost of accessing memory; T_D : cost of accessing disk
- P_{miss} : probability of a miss (0.0 to 1.0); you always pay T_M
- Example: $T_M = 100$ ns, $T_D = 10$ ms, 10% miss rate $\rightarrow$ about 1 ms

10 references to pages 0–9; only page 3 is not in memory

p0 hit	p1 hit	p2 hit	p3 miss	p4 hit	p5 hit	p6 hit	p7 hit	p8 hit	p9 hit
-----------	-----------	-----------	------------	-----------	-----------	-----------	-----------	-----------	-----------

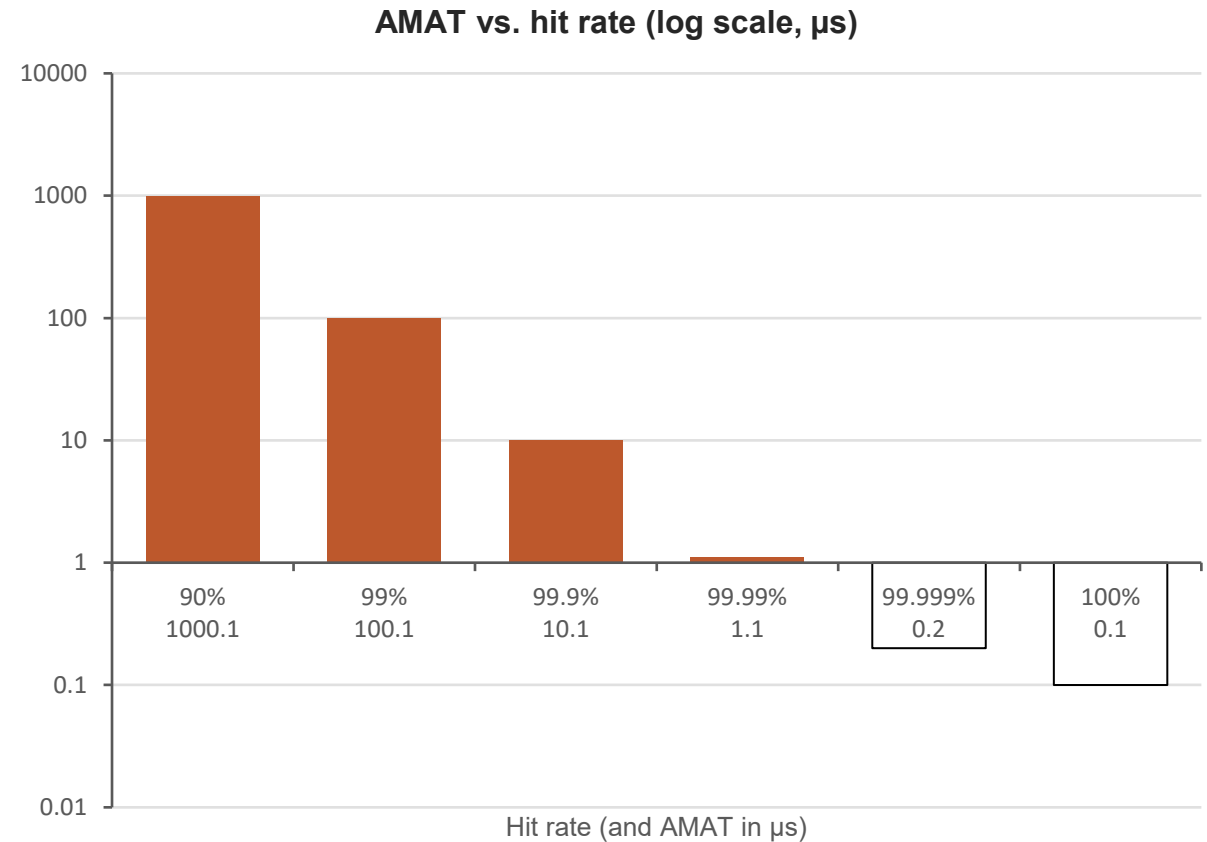
hit rate 90% · miss rate $P_{miss} = 0.1$

$$AMAT = T_M + (P_{miss} \times T_D)$$

$$\begin{aligned} T_M &\approx 100 \text{ ns (memory)} & T_D &\approx 10 \text{ ms (disk)} \\ AMAT &= 100 \text{ ns} + 0.1 \times 10 \text{ ms} \\ &\approx 1.0001 \text{ ms} \approx 1 \text{ ms} \end{aligned}$$

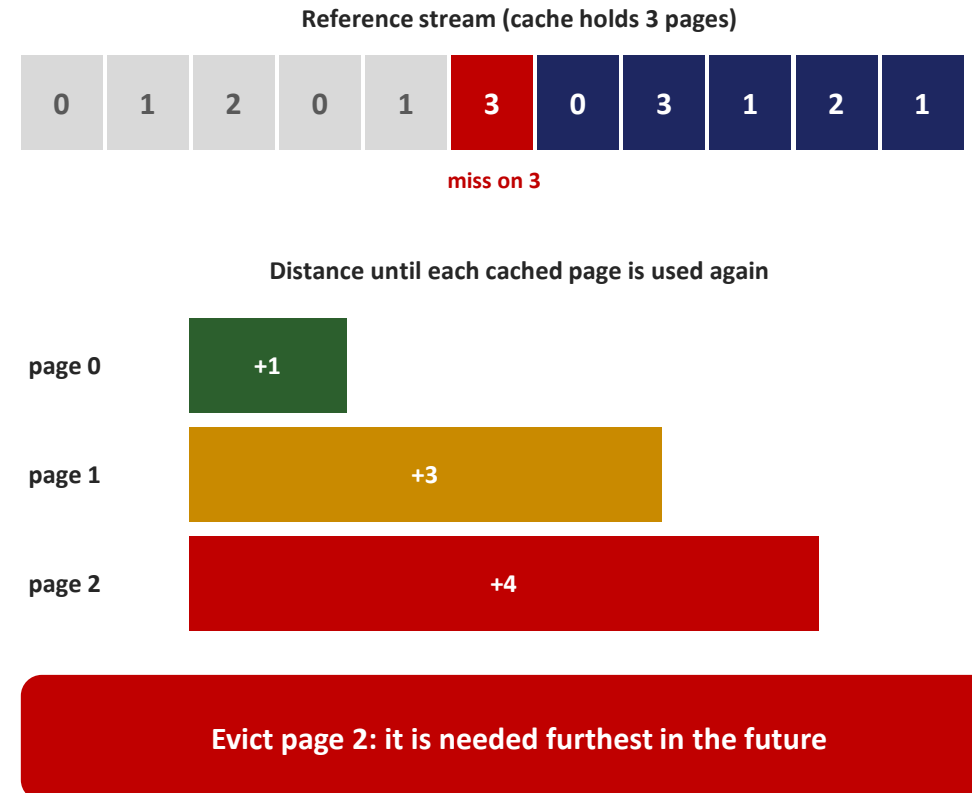
A Tiny Miss Rate Dominates AMAT

- 90% hit rate $\rightarrow$ AMAT $\approx$ 1 ms
- 99.9% hit rate ($P_{\text{miss}} = 0.001$) $\rightarrow$ AMAT $\approx$ 10.1 μs , about 100 $\times$ faster
- As the hit rate approaches 100%, AMAT approaches 100 ns
- Disk is so slow that even a tiny miss rate dominates AMAT**
- We must avoid as many misses as possible $\rightarrow$ a smart policy



The Optimal Replacement Policy

- To judge a policy, compare it to the best possible one
- Belady's optimal (MIN) policy gives the fewest misses overall
- **Rule: replace the page that will be accessed furthest in the future**
- Intuition: all other cached pages will be needed before that one
- Difficult to implement: it requires knowing the future
- **Tip: comparing against optimal is very useful in studies**



Tracing the Optimal Policy

- Reference stream: 0, 1, 2, 0, 1, 3, 0, 3, 1, 2, 1; cache holds 3 pages
- First three accesses are compulsory (cold-start) misses
- Miss on 3: evict 2 (its next use is furthest away)
- Miss on 2 later: evict 3 (page 0 would have been fine too)
- 6 hits and 5 misses → 54.5% hit rate

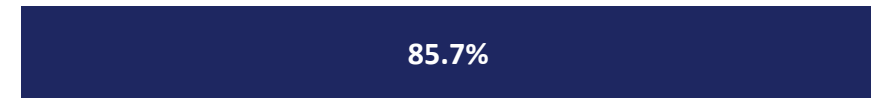
Access	Hit/Miss	Evict	Cache state
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	2	0, 1, 3
0	Hit		0, 1, 3
3	Hit		0, 1, 3
1	Hit		0, 1, 3
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

6 hits / 5 misses = 54.5%

Optimal: Results and Types of Misses

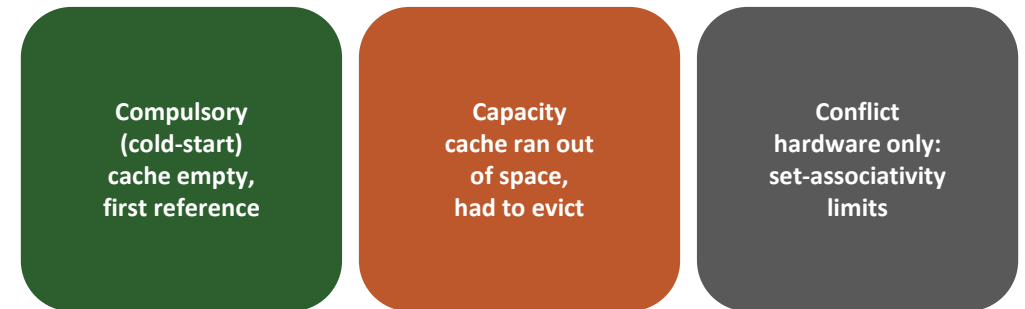
- Hit rate = hits / (hits + misses) = 6 / 11 = 54.5%
- Ignoring compulsory misses (first touch of each page): 85.7%
- Optimal is unrealizable: the future is unknown in a general OS
- It serves as a comparison point for real policies
- **The Three C's: compulsory, capacity, conflict**
 - An OS page cache is fully associative → no conflict misses

Hit rate of the optimal policy on the trace



ignoring the 4 compulsory misses: 6 hits / 7 references

The Three C's of cache misses



OS page caches are fully associative → no conflict misses

A Simple Policy: FIFO

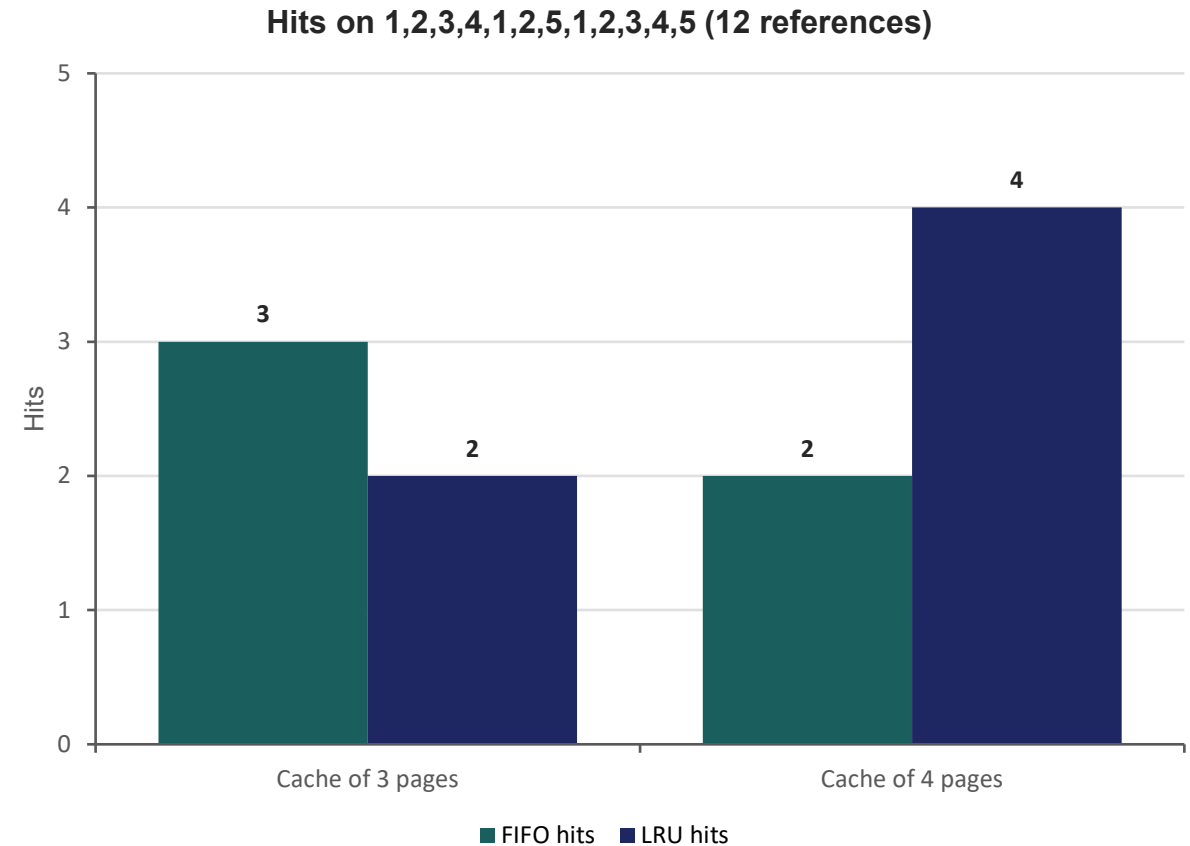
- Early systems used simple policies to avoid complexity
- FIFO: pages are queued as they enter; evict the first-in page
- Strength: very simple to implement**
- On our stream: 4 hits and 7 misses → 36.4% hit rate
- 57.1% excluding compulsory misses — notably worse than optimal
- FIFO can't judge importance: page 0 is evicted despite repeated use**

Access	Hit/Miss	Evict	Cache state
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0
1	Miss	2	3, 0, 1
2	Miss	3	0, 1, 2
1	Hit		0, 1, 2

cache shown first-in → last-in; 4 hits / 7 misses = 36.4%

Belady's Anomaly and the Stack Property

- Reference stream: 1, 2, 3, 4, 1, 2, 5, 1, 2, 3, 4, 5 under FIFO
- Expectation: a larger cache should never hurt the hit rate
- FIFO with 3 pages: 3 hits — with 4 pages: only 2 hits!**
- This oddity is called Belady's Anomaly
- LRU has the stack property: a size $N+1$ cache contains a size N cache**
 - So LRU's hit rate can only stay the same or improve; FIFO and Random lack it



Another Simple Policy: Random

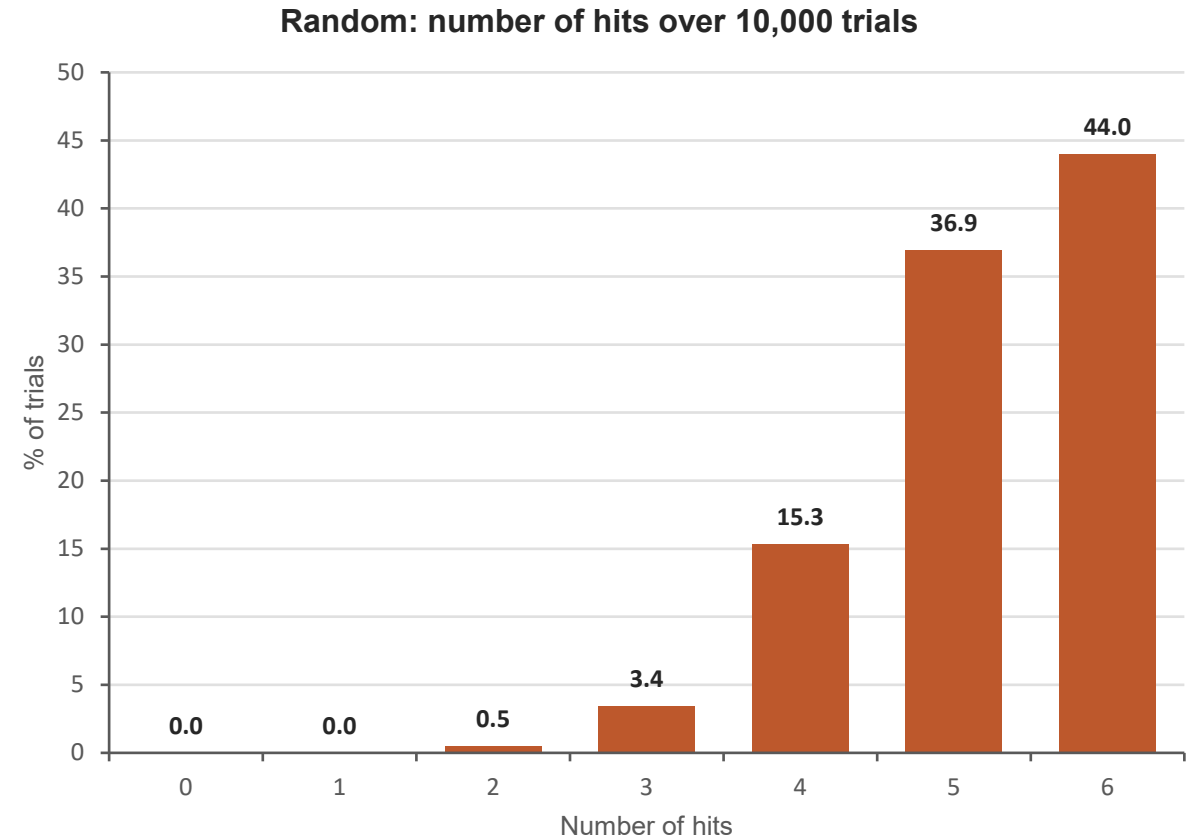
- Random simply picks a random page to replace under memory pressure
- Similar to FIFO: simple to implement, but not very intelligent
- Its result depends entirely on the luck of the draw
- In this run: 5 hits → a little better than FIFO, a little worse than optimal
- Next: run the experiment thousands of times to see the distribution

Access	Hit/Miss	Evict	Cache state
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		0, 1, 2
1	Hit		0, 1, 2
3	Miss	0	1, 2, 3
0	Miss	1	2, 3, 0
3	Hit		2, 3, 0
1	Miss	3	2, 0, 1
2	Hit		2, 0, 1
1	Hit		2, 0, 1

one lucky/unlucky run: 5 hits / 6 misses

How Lucky Is Random? 10,000 Trials

- Run Random 10,000 times, each with a different random seed
- Sometimes (just over 40% of the time) Random matches optimal: 6 hits
- Sometimes it does much worse: 2 hits or fewer
- Outcome depends on the luck of the draw
- Random is simple but offers no guarantee of good behavior



Using History: LRU and LFU

- FIFO and Random may evict an important page that is about to be reused
- Lean on the past to guess the future, as in scheduling
- **LFU evicts the least-frequently-used page; LRU the least-recently-used**
- They rely on the principle of locality (spatial and temporal)
- Locality is a heuristic, not a hard law
- Opposites (MFU, MRU) usually work badly: they ignore locality

Recency
→ LRU: evict the
least-recently used

Frequency
→ LFU: evict the
least-frequently used

Why history works: the principle of locality

Spatial locality
if page P is used,
P-1 and P+1
likely follow

Temporal locality
recently used pages
are likely to be
used again soon

Opposites (MRU, MFU) usually perform badly:
they ignore the locality programs exhibit

Tracing the LRU Policy

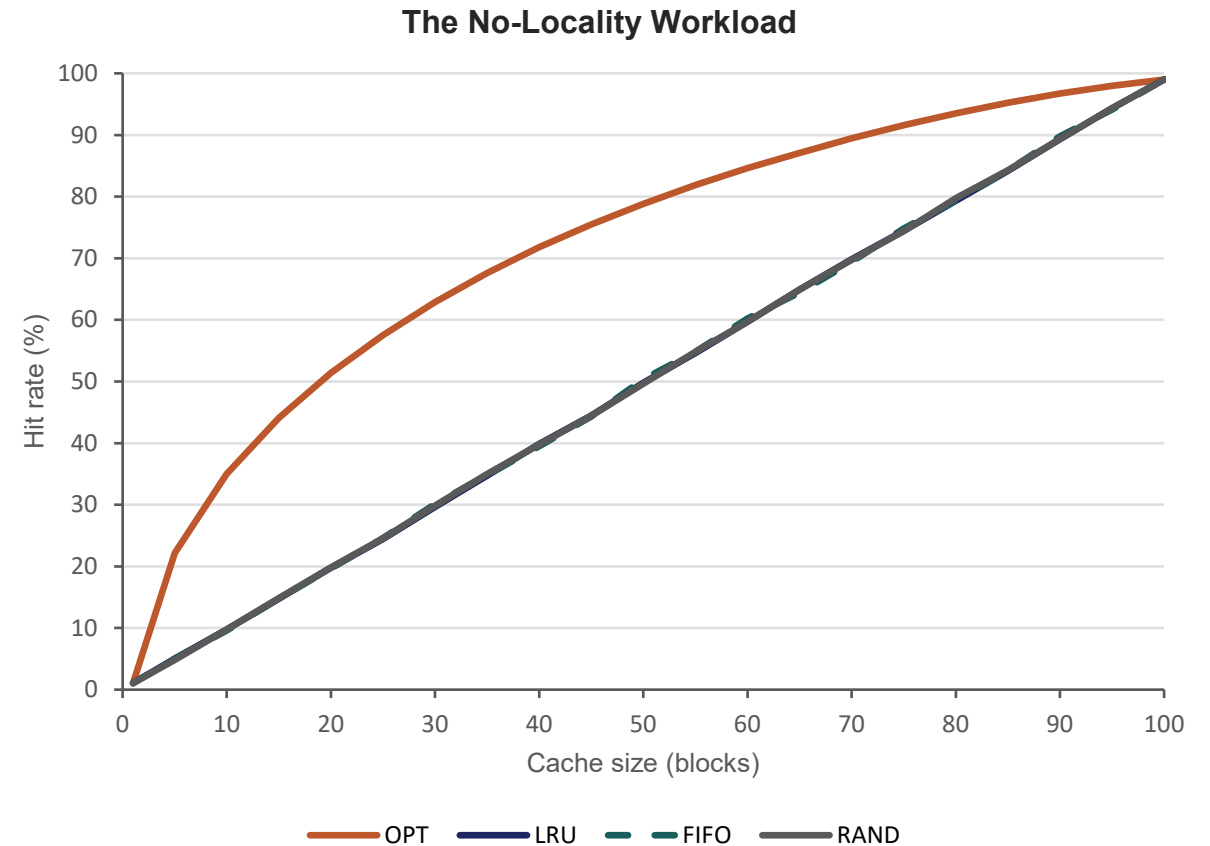
- LRU evicts the page that was used least recently
- Miss on 3: evict 2, because 0 and 1 were used more recently
- Miss on 2: evict 0, because 1 and 3 were used more recently
- Both decisions, based on history, turn out to be correct
- 6 hits → matches optimal on this stream (a favorable example)

Access	Hit/Miss	Evict	Cache state
0	Miss		0
1	Miss		0, 1
2	Miss		0, 1, 2
0	Hit		1, 2, 0
1	Hit		2, 0, 1
3	Miss	2	0, 1, 3
0	Hit		1, 3, 0
3	Hit		1, 0, 3
1	Hit		0, 3, 1
2	Miss	0	3, 1, 2
1	Hit		3, 2, 1

cache shown LRU → MRU; 6 hits / 5 misses = 54.5%

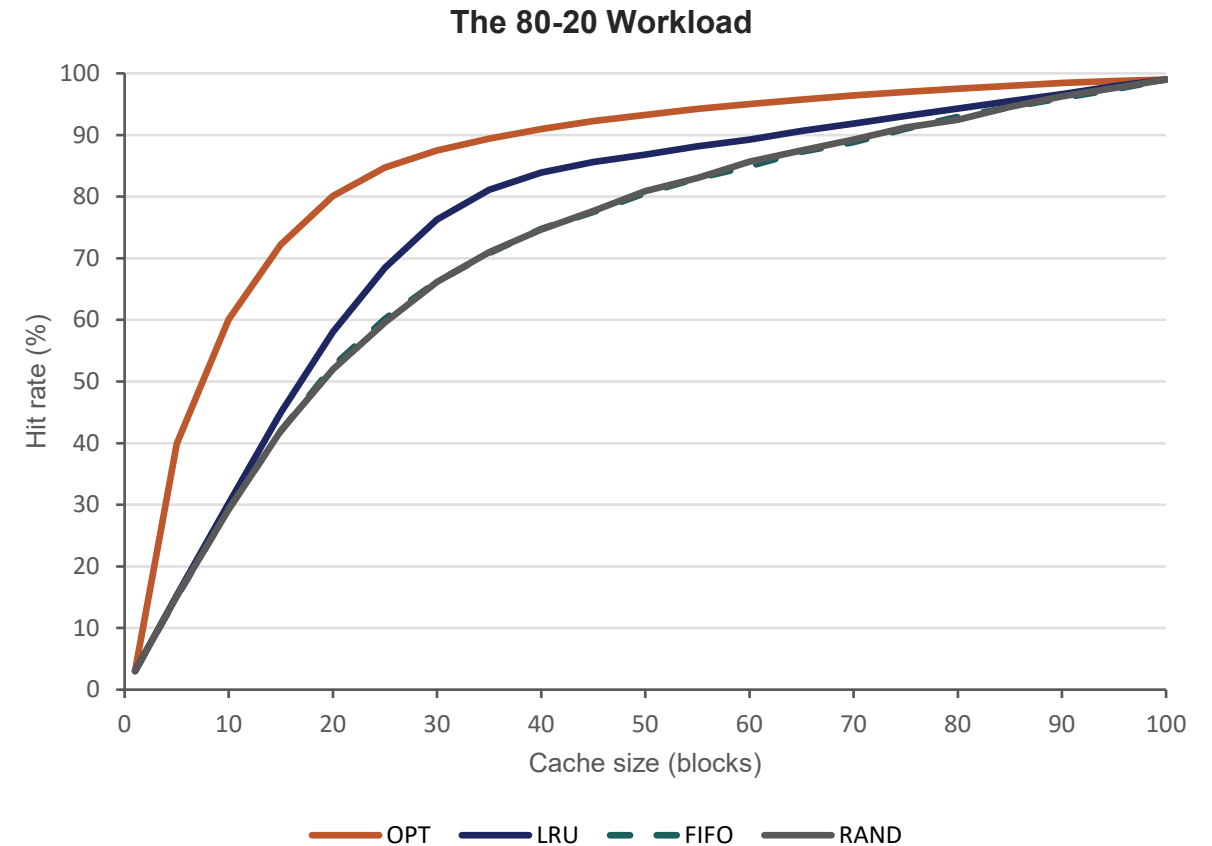
Workload 1: No Locality

- 100 unique pages, 10,000 references, each to a random page
- Vary the cache size from 1 to 100 pages
- LRU, FIFO and Random perform the same: hit rate set by cache size
- When the cache holds all 100 pages, every policy reaches 100%
- Optimal does noticeably better — but needs the future



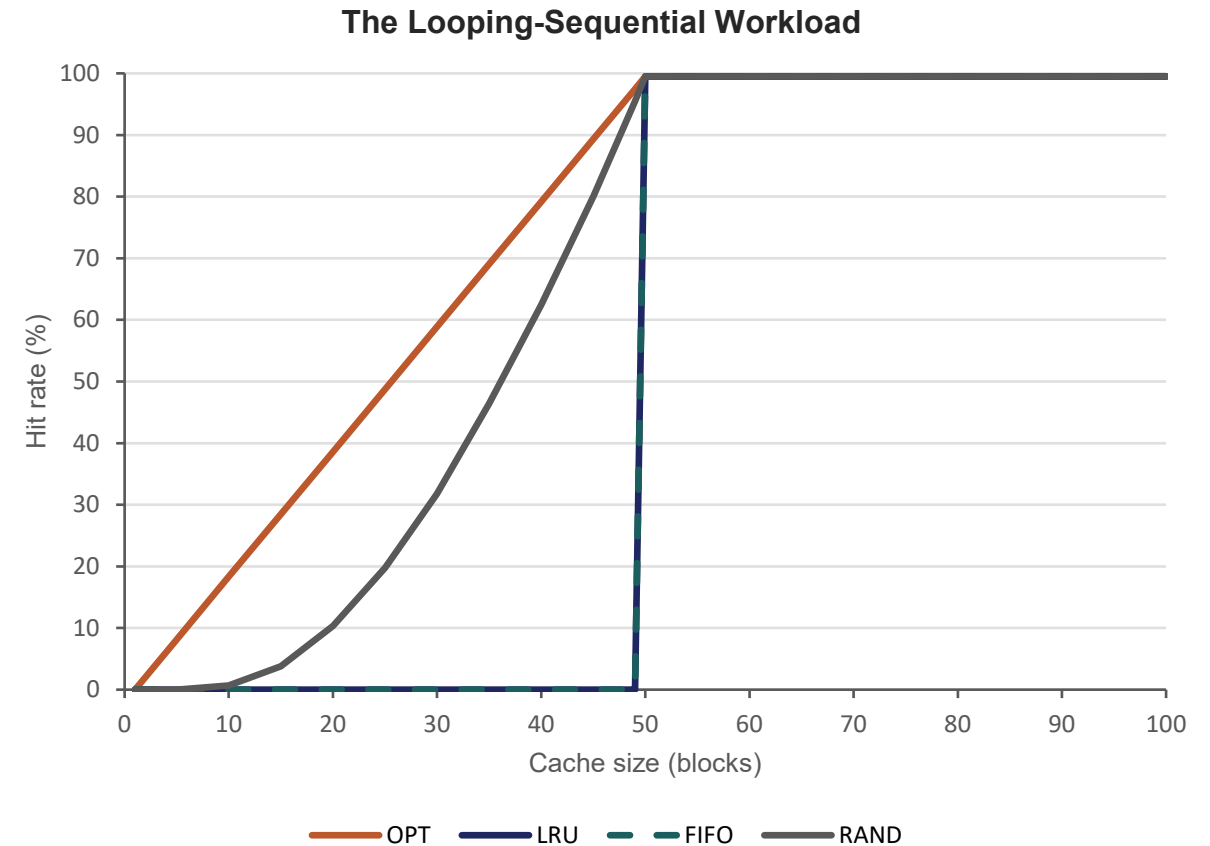
Workload 2: The 80-20 Workload

- 80% of references go to 20% of the pages (the hot pages)
- The other 20% of references go to the remaining 80% (cold pages)
- Random and FIFO do reasonably well; LRU does better
- LRU is more likely to hold onto the hot pages
- Optimal still wins → LRU's history isn't perfect
- Is LRU's gain a big deal? It depends on how costly a miss is



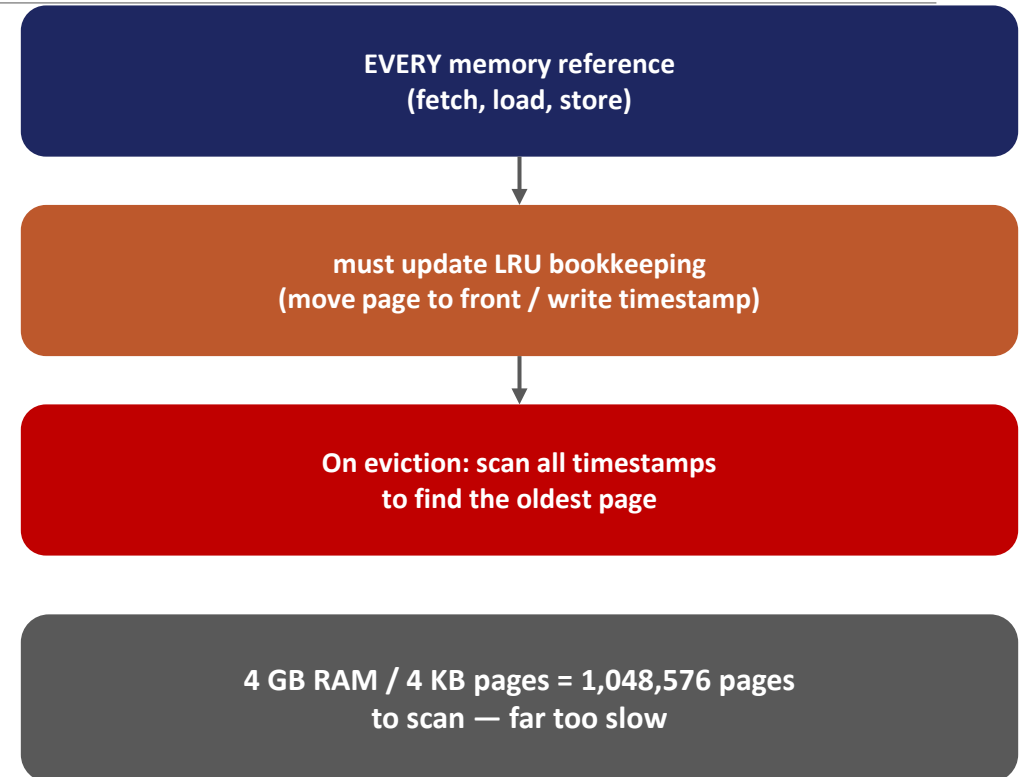
Workload 3: Looping Sequential

- Refer to 50 pages in order (0 ... 49), then loop: 10,000 accesses
- Common in many applications, including databases
- Worst case for both LRU and FIFO**
- They evict the older pages, which the loop will need soonest
- Even a cache of 49 pages gives a 0% hit rate for LRU and FIFO
- Random fares better; it has no weird corner-case behavior



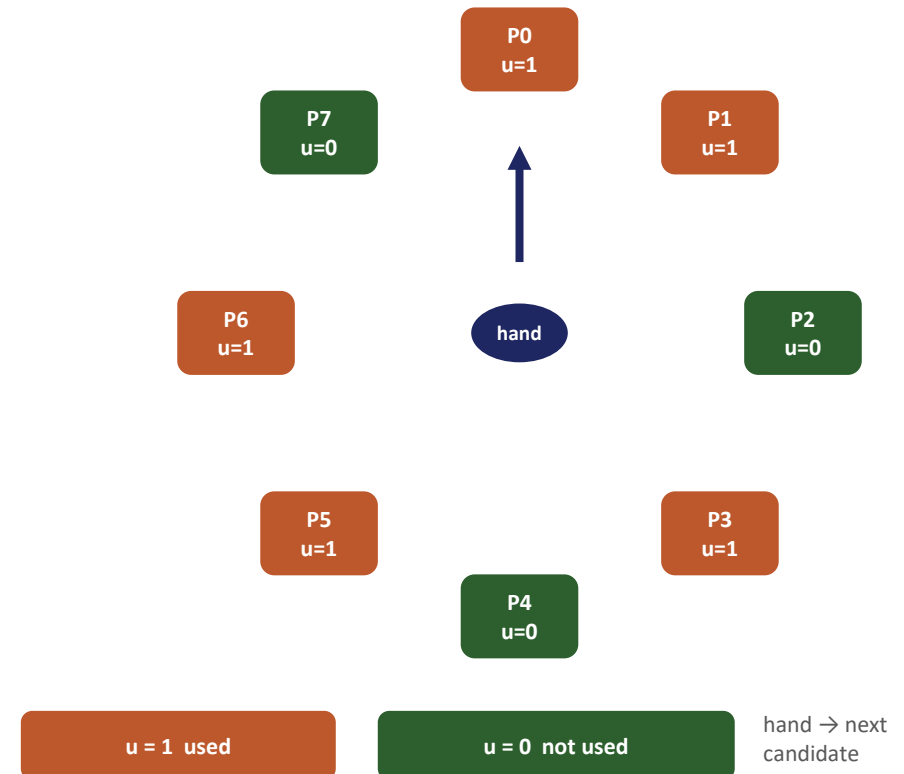
Implementing LRU Is Costly

- Perfect LRU needs bookkeeping on EVERY memory reference
- Move the page to the front of a list, or write a timestamp
- FIFO touches its list only on insert or evict — far cheaper
- Hardware could stamp a time field on each access
- But finding the oldest page means scanning all pages: 4 GB / 4 KB = 1M
- **Crux: can we approximate LRU cheaply?**



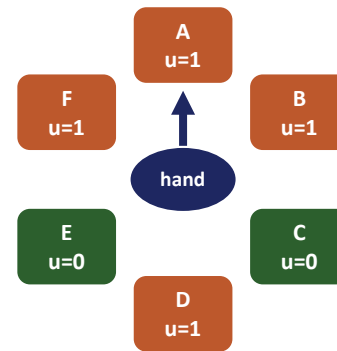
Approximating LRU: The Use Bit

- Hardware sets a use (reference) bit to 1 on every read or write
- Hardware never clears it; clearing is the OS's job
- **Clock algorithm: arrange pages in a circular list with a clock hand**
- On replacement, look at the page the hand points to
 - use bit 1 → recently used: clear it, advance the hand
 - use bit 0 → not recently used: evict it

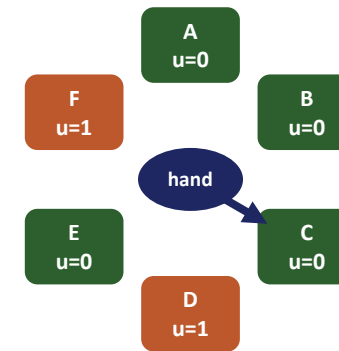


The Clock Algorithm in Action

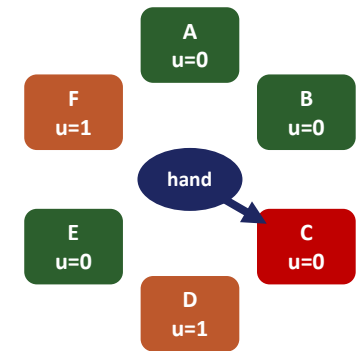
- Six pages A–F in a circle; the hand starts at A
- A has $u = 1$ → clear it and move on
- B has $u = 1$ → clear it and move on
- C has $u = 0$ → C is the victim
- Used pages get a second chance; no scan of all of memory is needed
- Any scheme that periodically clears use bits would also work



1. start: hand at A



2. clear A and B
(both were used)



3. C has $u = 0$
→ evict C

Used pages get a second chance:
their bit is cleared, the hand moves on.
The first page found with $u = 0$ is the victim.

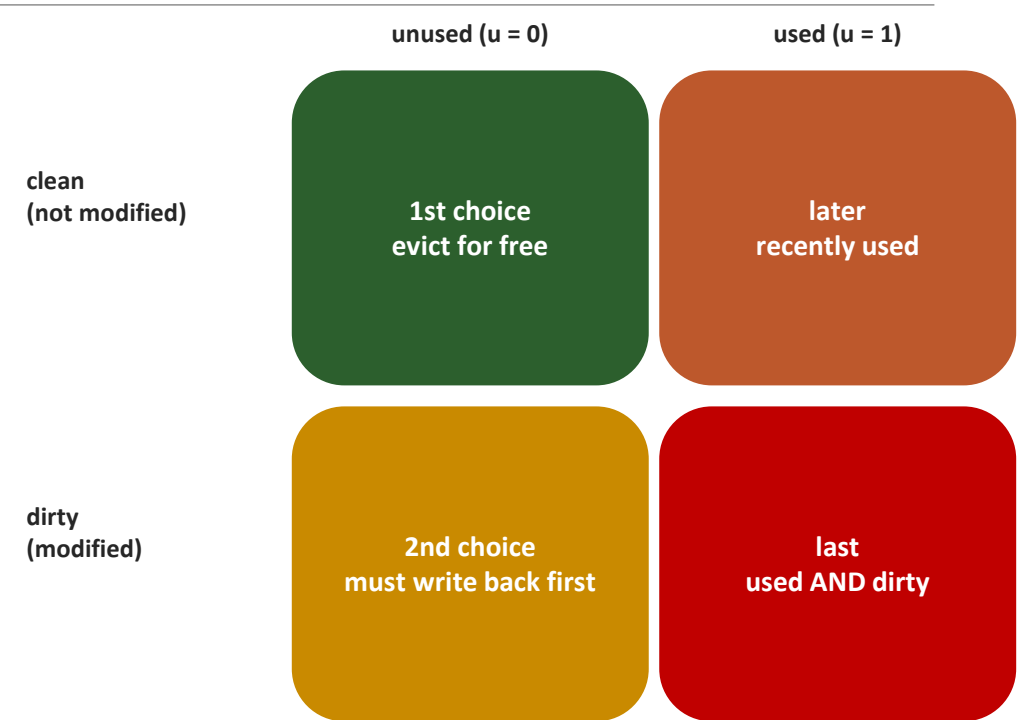
Clock Performance on the 80-20 Workload

- Variant tested: scan pages starting at a random page
- If the reference bit is 1: clear it and keep scanning
- If it is 0: choose that page as the victim
- Clock does not quite match perfect LRU
- **But it does better than policies that ignore history (FIFO, Random)**



Considering Dirty Pages

- A common clock tweak: consider whether a page has been modified
- Dirty page: must be written back to disk before eviction — expensive
- Clean page: eviction is free; the frame is simply reused
- Hardware supplies a modified (dirty) bit, set whenever the page is written
- Prefer evicting clean pages over dirty ones
- Scan for pages that are unused AND clean first; then unused but dirty



Clean pages can simply be dropped; dirty pages cost an extra disk write.

Other VM Policies

- Replacement is not the only policy, though it may be the most important
- **Page selection: when to bring a page into memory**
 - Demand paging: on access, “on demand” (the default)
 - Prefetching: bring in P+1 when P is loaded, if success is likely
- **Page writes: how to write pages out**
 - Clustering/grouping: collect pending writes and write them together

Demand paging

bring a page in when it is accessed, on demand

Prefetching

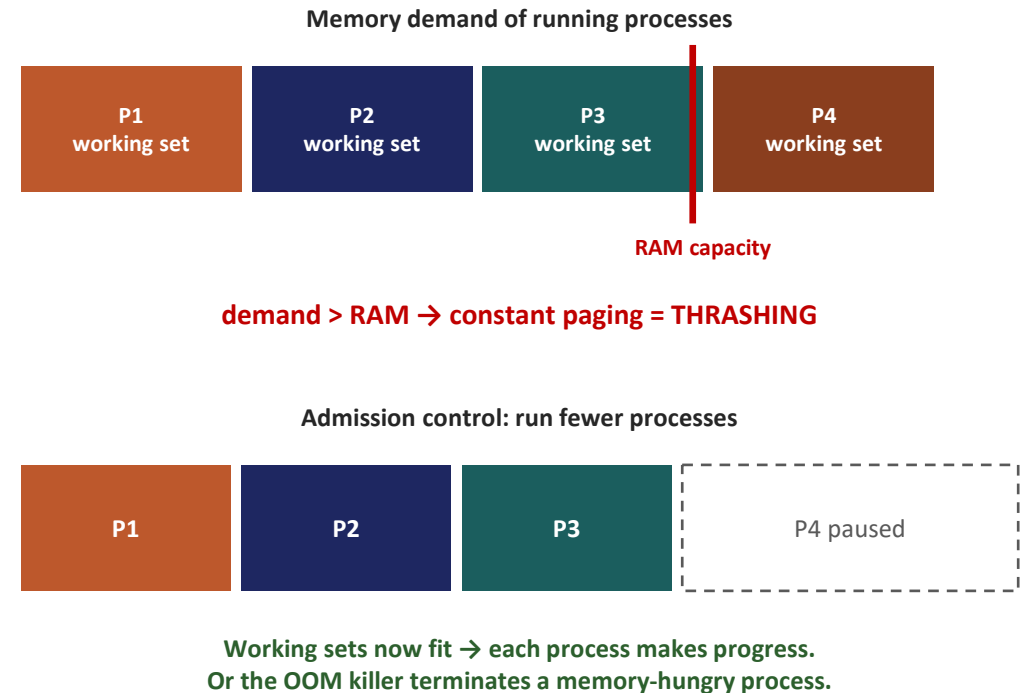
guess ahead: if code page P is loaded, also load P+1 (only when success is likely)

Clustering / grouping

collect pending writes and write many pages in one efficient disk write

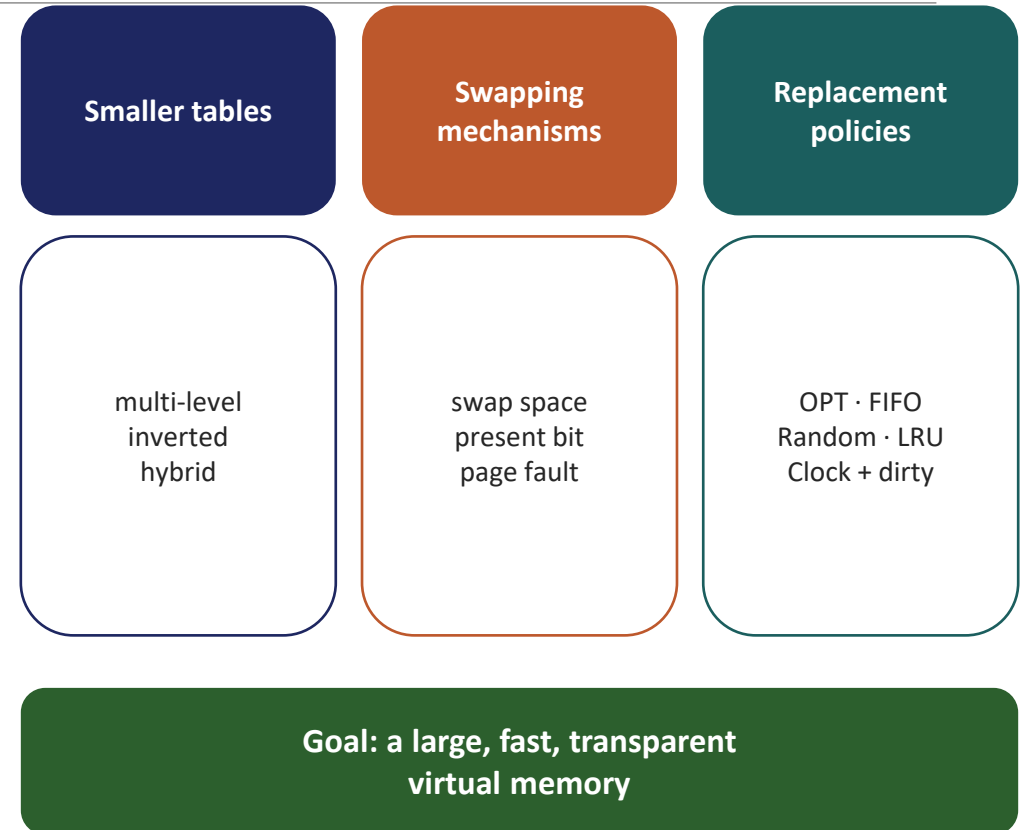
Thrashing

- What if memory demand of running processes exceeds physical memory?
- The system constantly pages: a condition called thrashing
- Working sets: the pages a process is actively using
- Admission control: run fewer processes so their working sets fit**
 - Better to do less work well than everything poorly
- Some Linux versions run an out-of-memory killer to end a hungry process



Summary: Putting It Together

- **Smaller tables: multi-level page tables allocate only what is used**
- Present bit + page faults let the OS use disk as a larger, slower memory
- Replacement policy is the key decision once memory is full
- LRU-like policies exploit locality; Clock approximates LRU cheaply
- Prefer clean pages; avoid thrashing with admission control
- Newer fast storage (e.g., flash SSDs) has revived interest in replacement



Thank You!